ML1516 - CLOUD COMPUTING FOR MACHINE LEARNING
UNIT IV - CLOUD PLATFORMS AND DEVELOPMENTS

Hadoop – MapReduce – Google App Engine - Amazon web services – Compute with EC2 – SQL Database with AWS RDS – NoSQL Database with DocumentDB – Cloud Storage Providers - S3 – Storage types in S3 – Triggers with Lambda – Queuing with SQS – Deploying a simple NodeJS application with EC2

Hadoop

- Hadoop is an open source implementation of MapReduce coded and released in Java(rather than C) by Apache.
- The Hadoop implementation of MapReduce uses the Hadoop Distributed File System(HDFS) as its underlying layer rather than GFS.
- The Hadoop core is divided into two fundamental layers:
    - MapReduce engine
    - HDFS
- The MapReduce engine is the computation engine running on top of HDFS as its datastorage manager.
- HDFS is a distributed file system inspired by GFS that organizes files and stores theirdata on a distributed computing system.
- HDFS Architecture: HDFS has a master/slave architecture containing a singleNameNode as the master and a number of DataNodes as workers (slaves).
- To store a file in this architecture, HDFS splits the file into fixed-size blocks (e.g., 64 MB)and stores them on workers (DataNodes).
- The mapping of blocks to DataNodes is determined by the NameNode.
- The NameNode (master) also manages the file system's metadata and namespace.
- In such systems, the namespace is the area maintaining the metadata and metadata refers to all the information stored by a file system that is needed for overall management of all files.
- For example, NameNode in the metadata stores all information regarding the location ofinput splits/blocks in all DataNodes.

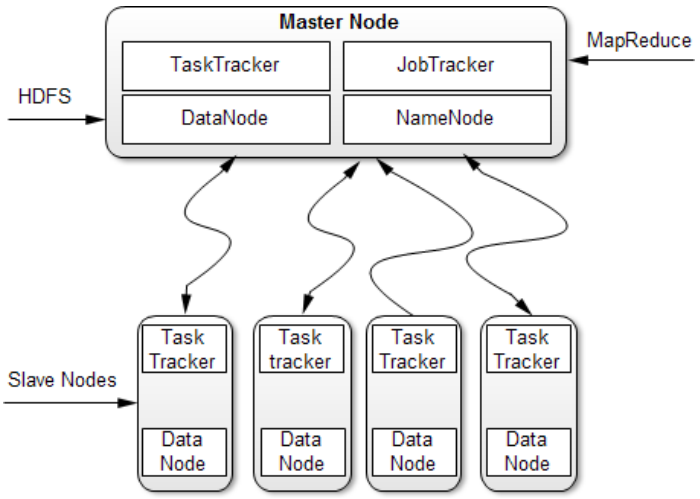
- Each DataNode, usually one per node in a cluster, manages the storage attached to the node. Each DataNode is responsible for storing and retrieving its file blocks.
- HDFS Features: Distributed file systems have special requirements, such as performance, scalability, concurrency control, fault tolerance and security requirements, to operate efficiently.
- However, because HDFS is not a general purpose file system, as it only executes specific types of applications, it does not need all the requirements of a general distributed file system.
- One of the main aspects of HDFS is its fault tolerance characteristic. Since Hadoop is designed to be deployed on low-cost hardware by default, a hardware failure in this system is considered to be common rather than an exception.
- Hadoop considers the following issues to fulfill reliability requirements of the file system
    - Block replication: To reliably store data in HDFS, file blocks are replicated in thissystem. The replication factor is set by the user and is three by default.
    - Replica placement: The placement of replicas is another factor to fulfill the desired fault tolerance in HDFS.
    - Heartbeat and Block report messages: Heartbeats and Block reports are periodic messages sent to the NameNode by each DataNode in a cluster.
    - Applications run on HDFS typically have large data sets, individual files are broken into large blocks (e.g., 64 MB) to allow HDFS to decrease the amount of metadata storage required per file.
    - This provides two advantages:
        - The list of blocks per file will shrink as the size of individual blocksincreases.
        - Keeping large amounts of data sequentially within a block provides faststreaming reads of data.
- HDFS Operation: The control flow of HDFS operations such as write and read can properly highlight roles of the NameNode and DataNodes

in the managing operations

- To read a file in HDFS, a user sends an “open” request to the NameNode to get the location of file blocks.
- For each file block, the NameNode returns the address of a set of DataNodes containing replica information for the requested file.
- The number of addresses depends on the number of block replicas. Uponreceiving such information, the user calls the read function to connect to the closest DataNode containing the first block of the file.
- After the first block is streamed from the respective DataNode to the user, the established connection is terminated and the same process is repeated for all blocks of the requested file until the whole file is streamed to the user.
- To write a file in HDFS, a user sends a “create” request to the NameNode to create a new file in the file system namespace.
- If the file does not exist, the NameNode notifies the user and allows him to start writing data to the file by calling the write function.
- The first block of the file is written to an internal queue termed the data queue while a data streamer monitors its writing into a DataNode.
- Since each file block needs to be replicated by a predefined factor, the data streamer first sends a request to the NameNode to get a list of suitable DataNodes to store replicas of the first block.
- The steamer then stores the block in the first allocated DataNode.
- Afterward, the block is forwarded to the second DataNode by the first DataNode.
- The process continues until all allocated DataNodes receive a replica of the first block from the previous DataNode.
- Once this replication process is finalized, the same process starts for the second block and continues until all blocks of the file are stored and replicated on the filesystem.

MapReduce

- The topmost layer of Hadoop is the MapReduce engine that manages the data flow and control flow of MapReduce jobs over distributed computing systems.

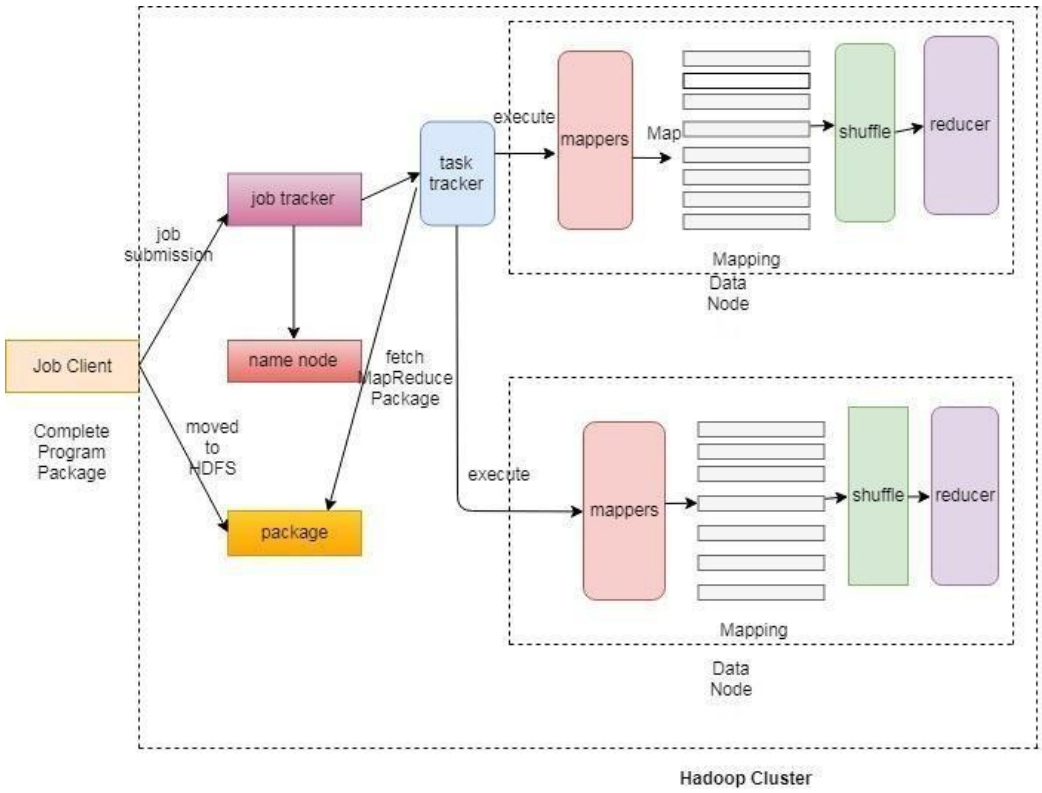


HDFS and MapReduce Architecture

- Figure shows the MapReduce engine architecture cooperating with HDFS.
- Similar to HDFS, the MapReduce engine also has a master/slave architecture consisting of a single JobTracker as the master and a number of TaskTrackers as the slaves (workers).
- The JobTracker manages the MapReduce job over a cluster and is responsible formonitoring jobs and assigning tasks to TaskTrackers.
- The TaskTracker manages the execution of the map and/or reduce tasks on a singlecomputation node in the cluster.
- Each TaskTracker node has a number of simultaneous execution slots, each executingeither a map or a reduce task.
- Slots are defined as the number of simultaneous threads supported by CPUs of theTaskTracker node.
- For example, a TaskTracker node with N CPUs, each supporting M threads, has M * Nsimultaneous execution slots.
- It is worth noting that each data block is processed by one map task running on a singleslot.
- Therefore, there is a one to one correspondence between map tasks in a TaskTrackerand data blocks in the respective DataNode.

Running a Job in Hadoop

- Three components contribute in running a job in this system:
    - User node
    - JobTracker
    - TaskTrackers
- The data flow starts by calling the runJob (conf) function inside a user program running on the user node, in which conf is an object containing some tuning parameters for the MapReduce framework and HDFS.
- The runJob (conf) function and conf are comparable to the MapReduce (Spec, &Results)function and Spec in the first implementation of MapReduce by Google.
- Figure depicts the data flow of running a MapReduce job in Hadoop.



Data flow in Hadoop

- Job Submission Each job is submitted from a user node to the JobTracker node that might be situated in a different node within the cluster through the following procedure:
    - A user node asks for a new job ID from the JobTracker and computes input filesplits.

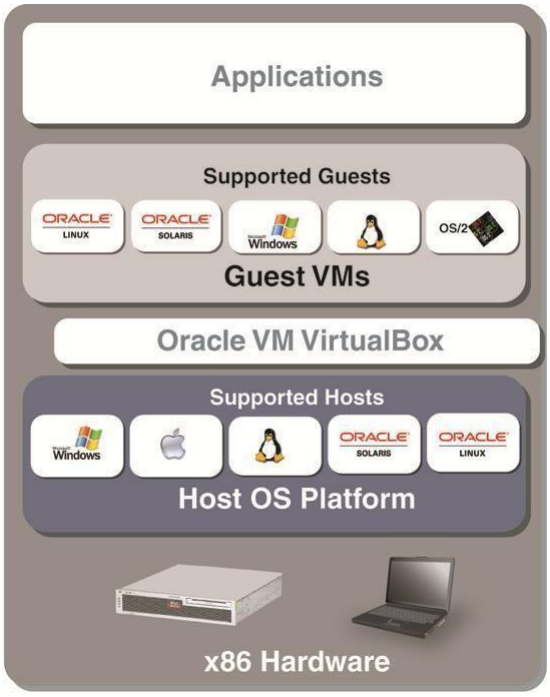
- The user node copies some resources, such as the job's JAR file, configuration file, and computed input splits, to the JobTracker's file system.
- The user node submits the job to the JobTracker by calling the submitJob() function.
- Task assignment The JobTracker creates one map task for each computed input split by the user node and assigns the map tasks to the execution slots of the TaskTrackers.
  - The JobTracker considers the localization of the data when assigning themap tasks to the TaskTrackers.
  - The JobTracker also creates reduce tasks and assigns them to theTaskTrackers.
  - The number of reduce tasks is predetermined by the user, and there isno locality consideration in assigning them.
- Task execution The control flow to execute a task (either map or reduce) startsinside the TaskTracker by copying the job JAR file to its file system.
- Instructions inside the job JAR file are executed after launching a Java VirtualMachine (JVM) to run its map or reduce task.
- Task running check A task running check is performed by receiving periodicheartbeat messages to the JobTracker from the TaskTrackers.
- Each heartbeat notifies the JobTracker that the sending TaskTracker is alive, andwhether the sending TaskTracker is ready to run a new task.

Virtual Box

- Oracle VM VirtualBox is a cross platform virtualization application.
- For one thing, it installs on the existing Intel or AMD-based computers, whether they arerunning Windows, Mac OS X, Linux, or Oracle Solaris operating systems (OSes).
- Secondly, it extends the capabilities of existing computer so that it can run multiple OSes, inside multiple virtual machines, at the same time.As an example, the end user can run Windows and Linux on your Mac, run Windows Server 2016 on your Linux server, run Linux on your Windows PC, and so

on, all alongside the existing applications.The user can install and run as many virtual machines.

- The only practical limits are disk space and memory.
- Oracle VM VirtualBox is deceptively simple yet also very powerful.
- It can run everywhere from small embedded systems or desktop class machines all the way up to datacenter deployments and even Cloud environments.
- Virtual Box is created by Innotek and it was acquired by Sun Microsystems. In 2010,Virtual Box was acquired by Oracle.



Architecture of Virtual Box

- Virtual Box supported in Windows, macOS. Linux, Solaris and Open Solaris.
- The user can independently configure each VM and run it under a choice of software- based virtualization or hardware assisted virtualization if the underlying host hardware supports this.
- The host OS and guest OSs and applications can communicate with each other througha number of mechanisms including a common clipboard and a virtualized network facility.
- Guest VMs can also directly communicate with each other if configured to do so.
- The software based virtualization was dropped starting with VirtualBox 6.1.

In earlier versions the absence of hardware assisted virtualization, VirtualBox adopts astandard software-based virtualization approach.

- This mode supports 32 bit guest OSs which run in rings 0 and 3 of the Intel ring architecture.
    - The system reconfigures the guest OS code, which would normally run in ring 0, to execute in ring 1 on the host hardware.
    - Because this code contains many privileged instructions which cannot run natively in ring 1, VirtualBox employs a Code Scanning and Analysis Manager (CSAM) to scan the ring 0 code recursively before its first execution to identify problematic instructions and then calls the Patch Manager (PATM) to perform in- situ patching.
    - This replaces the instruction with a jump to a VM-safe equivalent compiled code fragment in hypervisor memory.
    - The guest user mode code, running in ring 3, generally runs directly on the host hardware in ring 3.
- In both cases, VirtualBox uses CSAM and PATM to inspect and patch the offendinginstructions whenever a fault occurs.
- VirtualBox also contains a dynamic recompiler, based on QEMU to recompile any realmode or protected mode code entirely.
- Hardware assisted virtualization is starting with version 6.1, VirtualBox only supports.
- VirtualBox supports both Intel VT-X and AMD-V hardware assisted virtualization.
- Making use of these facilities, VirtualBox can run each guest VM in its own separateaddress-space.
- The guest OS ring 0 code runs on the host at ring 0 in VMX non-root mode rather than inring 1.
- Until then, VirtualBox specifically supported some guests (including 64 bit guests, SMPguests and certain proprietary OSs) only on hosts with hardware-assisted virtualization
- The system emulates hard disks in one of three disk image formats:
    - VDI: This format is the VirtualBox-specific VirtualBox Disk Image and stores data in files bearing a ".vdi" .
    - VMDK: This open format is used by VMware products and stores data

in one or more files bearing ".vmdk" filename extensions. A single virtual hard disk may span several files.

- VHD: This format is used by Windows Virtual PC and Hyper-V and it is the native virtual disk format of the Microsoft Windows operating system. Data in this formatare stored in a single file bearing the ".vhd" filename extension.
- A VirtualBox virtual machine can, therefore, use disks previously created in VMware orMicrosoft Virtual PC, as well as its own native format.
- VirtualBox can also connect to iSCSI targets and to raw partitions on the host, usingeither as virtual hard disks.
- VirtualBox has supported Open Virtualization Format (OVF).
- By default, VirtualBox provides graphics support through a custom virtual graphics-card
- For an Ethernet network adapter, VirtualBox virtualizes these Network Interface Cards.
    - AMD PCnet PCI II
    - AMD PCnet-Fast III
    - Intel Pro/1000 MT Desktop
    - Intel Pro/1000 MT Server
    - Intel Pro/1000 T Server
    - Paravirtualized network adapter
- For a sound card, VirtualBox virtualizes Intel HD Audio.
- A USB controller is emulated so that any USB devices attached to the host can be seenin the guest.
- Oracle VM VirtualBox was designed to be modular and flexible.
- When the Oracle VM VirtualBox graphical user interface (GUI) is opened and a VM isstarted, at least the following three processes are running:
    - VBoxSVC is Oracle VM VirtualBox service process which always runs in the background. This process is started automatically by the first Oracle VM VirtualBox client process and exits a short time after the last client exits.
    - The first Oracle VM VirtualBox service can be the GUI,

VBoxManage,VBoxHeadless, the web service amongst others.

- The service is responsible for bookkeeping, maintaining the state of all VMs, andfor providing communication between Oracle VM VirtualBox components.

- Oracle VM VirtualBox comes with comprehensive support for third-party developers.
- The Main API of Oracle VM VirtualBox exposes the entire feature set of the virtualizationengine.
- The Main API is made available to C++ clients through COM on Windows hosts or XPCOM on other hosts. Bridges also exist for SOAP, Java and Python.

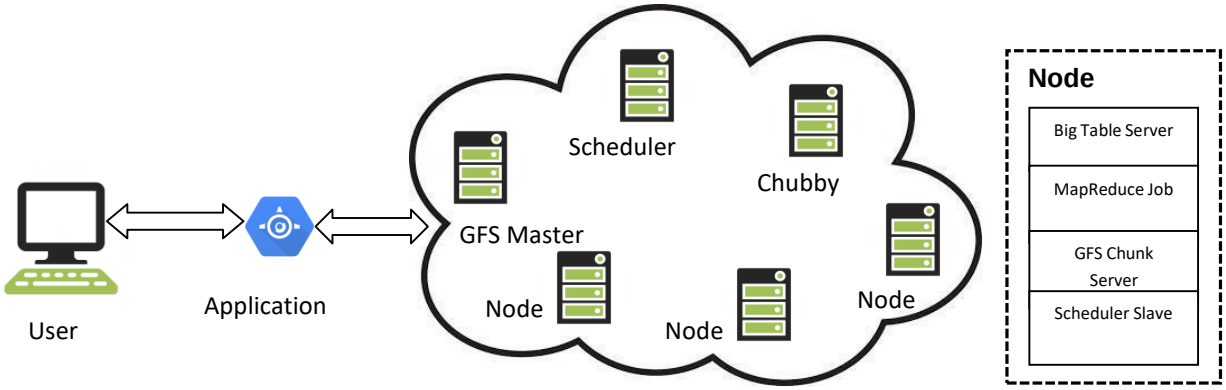
Google App Engine

- Google has the world's largest search engine facilities.
- The company has extensive experience in massive data processing that has led to new insights into data-center design and novel programming models that scale to incredible sizes.
- Google platform is based on its search engine expertise.
- Google has hundreds of data centers and has installed more than 460,000 servers worldwide.
- For example, 200 Google data centers are used at one time for a number of cloud applications.
- Data items are stored in text, images, and video and are replicated to tolerate faults or failures.
- Google's App Engine (GAE) which offers a PaaS platform supporting various cloud andweb applications.
- Google has pioneered cloud development by leveraging the large number of datacenters it operates.
- For example, Google pioneered cloud services in Gmail, Google Docs, and GoogleEarth, among other applications.

- These applications can support a large number of users simultaneously with HA.
- Notable technology achievements include the Google File System (GFS), MapReduce,BigTable, and Chubby.
- In 2008, Google announced the GAE web application platform which is becoming acommon platform for many small cloud service providers.
- This platform specializes in supporting scalable (elastic) web applications.
- GAE enables users to run their applications on a large number of data centersassociated with Google's search engine operations.

GAE Architecture

- Figure shows the major building blocks of the Google cloud platform which has beenused to deliver the cloud services highlighted earlier.
- GFS is used for storing large amounts of data.
- MapReduce is for use in application program development.
- Chubby is used for distributed application lock services.
- BigTable offers a storage service for accessing structured data.
- Users can interact with Google applications via the web interface provided by each application.
- Third-party application providers can use GAE to build cloud applications for providing services.
- The applications all run in data centers under tight management by Google engineers. Inside each data center, there are thousands of servers forming



different clusters

Google cloud platform

- Google is one of the larger cloud application providers, although it is fundamental service program is private and outside people cannot use the Google infrastructure to build their own service.
- The building blocks of Google's cloud computing application include the Google File System for storing large amounts of data, the MapReduce programming framework for application developers, Chubby for distributed application lock services, and BigTable as a storage service for accessing structural or semistructural data.
- With these building blocks, Google has built many cloud applications.
- A typical cluster configuration can run the Google File System, MapReduce jobs and BigTable servers for structure data.
- Extra services such as Chubby for distributed locks can also run in the clusters.
- GAE runs the user program on Google's infrastructure. As it is a platform running third- party programs, application developers now do not need to worry about the maintenanceof servers.
- GAE can be thought of as the combination of several software components.
- The frontend is an application framework which is similar to other web application frameworks such as ASP, J2EE and JSP.
- At the time of this writing, GAE supports Python and Java programming environments. The applications can run similar to web application containers.
- The frontend can be used as the dynamic web serving infrastructure which can provide the full support of common technologies.

Functional Modules of GAE

- The GAE platform comprises the following five major components.
- The GAE is not an infrastructure platform, but rather an application developmentplatform for users.
    - The datastore offers object-oriented, distributed, structured data

storage servicesbased on BigTable techniques. The datastore secures data management operations.

- The application runtime environment offers a platform for scalable web programming and execution. It supports two development languages: Python andJava.
- The software development kit (SDK) is used for local application development. The SDK allows users to execute test runs of local applications and upload application code.
- The administration console is used for easy management of user application development cycles, instead of for physical resource management.
- The GAE web service infrastructure provides special interfaces to guarantee flexible use and management of storage and network resources by GAE.

- Google offers essentially free GAE services to all Gmail account owners.The user can register for a GAE account or use your Gmail account name to sign up forthe service.
- The service is free within a quota.
- If the user exceeds the quota, the page instructs how to pay for the service. Then theuser can download the SDK and read the Python or Java guide to get started.
- Note that GAE only accepts Python, Ruby and Java programming languages.
- The platform does not provide any IaaS services, unlike Amazon, which offers IaaS andPaaS.
- This model allows the user to deploy user-built applications on top of the cloud infrastructure that are built using the programming languages and software tools supported by the provider (e.g., Java, Python).
- Azure does this similarly for .NET. The user does not manage the underlying cloudinfrastructure.
- The cloud provider facilitates support of application development, testing, and operationsupport on a well-defined service platform.

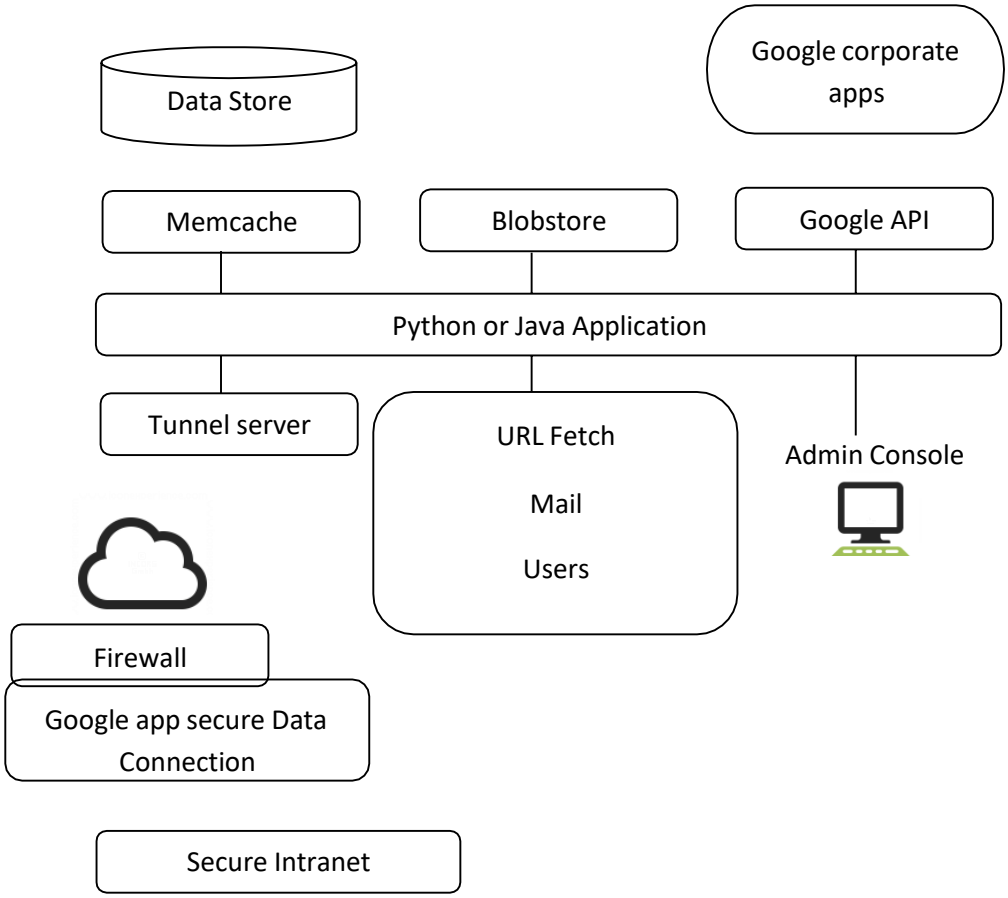
GAE Applications

- Best-known GAE applications include the Google Search Engine, Google Docs, GoogleEarth and Gmail.
- These applications can support large numbers of users simultaneously.
- Users can interact with Google applications via the web interface provided by eachapplication.
- Third party application providers can use GAE to build cloud applications for providingservices.
- The applications are all run in the Google data centers.
- Inside each data center, there might be thousands of server nodes to form differentclusters.
- Each cluster can run multipurpose servers.
- GAE supports many web applications.
- One is a storage service to store application specific data in the Google infrastructure.
- The data can be persistently stored in the backend storage server while still providing the facility for queries, sorting and even transactions similar to traditional database systems.
- GAE also provides Google specific services, such as the Gmail account service. Thiscan eliminate the tedious work of building customized user management components in web applications.

Programming Environment for Google App Engine

- Several web resources (e.g., http://code.google.com/appengine/) and specific books andarticles discuss how to program GAE.
- Figure summarizes some key features of GAE programming model for two supported languages: Java and Python.
- A client environment that includes an Eclipse plug-in for Java allows you to debug your GAE on your local machine.
- Also, the GWT Google Web Toolkit is available for Java web application developers. Developers can use this, or any other language using a JVM based interpreter or compiler, such as JavaScript or Ruby.

- Python is often used with frameworks such as Django and CherryPy, but Google also supplies a built in webapp Python environment.
- There are several powerful constructs for storing and accessing data.
- The data store is a NOSQL data management system for entities that can be, at most, 1 MB in size and are labeled by a set of schema-less properties.



Programming Environment of Google AppEngine

platform, while Python has a SQL-like query language called GQL.

- Queries can retrieve entities of a given kind filtered and sorted by the values of the properties.
- The data store is strongly consistent and uses optimistic concurrency control.
- An update of an entity occurs in a transaction that is retried a fixed number of times if other processes are trying to update the same entity simultaneously.
- The user application can execute multiple data store operations in a single transactionwhich either all succeed or all fail together.
- The data store implements transactions across its distributed network using entitygroups.

- A transaction manipulates entities within a single group.
- Entities of the same group are stored together for efficient execution of transactions.
- The user GAE application can assign entities to groups when the entities are created.
- The performance of the data store can be enhanced by in-memory caching using thememcache, which can also be used independently of the data store.
- Recently, Google added the blobstore which is suitable for large files as its size limit is 2GB.
- There are several mechanisms for incorporating external resources.
- The Google SDC Secure Data Connection can tunnel through the Internet and link yourintranet to an external GAE application.
- The URL Fetch operation provides the ability for applications to fetch resources andcommunicate with other hosts over the Internet using HTTP and HTTPS requests.
- There is a specialized mail mechanism to send e-mail from your GAE application.
- Applications can access resources on the Internet, such as web services or other data,using GAE's URL fetch service.
- The URL fetch service retrieves web resources using the same high-speed Google infrastructure that retrieves web pages for many other Google products.
- There are dozens of Google "corporate" facilities including maps, sites, groups, calendar, docs, and YouTube, among others.
- These support the Google Data API which can be used inside GAE.
- An application can use Google Accounts for user authentication. Google Accounts handles user account creation and sign-in, and a user that already has a Google account (such as a Gmail account) can use that account with your app.

- GAE provides the ability to manipulate image data using a dedicated Images service which can resize, rotate, flip, crop and enhance images. An application can perform tasks outside of responding to web requests.

- A GAE application is configured to consume resources up to certain limits or quotas.With quotas, GAE ensures that your application would not exceed your budget and that other applications running on GAE would not impact the performance of your app.

- In particular, GAE use is free up to certain quotas.

- GFS was built primarily as the fundamental storage service for Google's search engine.

- As the size of the web data that was crawled and saved was quite substantial, Google needed a distributed file system to redundantly store massive amounts of data on cheapand unreliable computers.

- In addition, GFS was designed for Google applications and Google applications were built for GFS.

- In traditional file system design, such a philosophy is not attractive, as there should be aclear interface between applications and the file system such as a POSIX interface.

- GFS typically will hold a large number of huge files, each 100 MB or larger, with files thatare multiple GB in size quite common. Thus, Google has chosen its file data block sizeto be 64 MB instead of the 4 KB in typical traditional file systems.

- The I/O pattern in the Google application is also special.

- Files are typically written once, and the write operations are often the appending data blocks to the end of files.

- Multiple appending operations might be concurrent.

- BigTable was designed to provide a service for storing and retrieving structured and semi structured data.

- BigTable applications include storage of web pages, per-user data, and geographic
- The scale of such data is incredibly large. There will be billions of URLs, and each URLcan have many versions, with an average page size of about 20 KB per version.
- The user scale is also huge.
- There are hundreds of millions of users and there will be thousands of queries per second.
- The same scale occurs in the geographic data, which might consume more than 100 TB of disk space.
- It is not possible to solve such a large scale of structured or semi structured data using acommercial database system.
- This is one reason to rebuild the data management system and the resultant system canbe applied across many projects for a low incremental cost.
- The other motivation for rebuilding the data management system is performance.
- Low level storage optimizations help increase performance significantly which is much harder to do when running on top of a traditional database layer.
- The design and implementation of the BigTable system has the following goals.
    - The applications want asynchronous processes to be continuously updating different pieces of data and want access to the most current data at all times.
    - The database needs to support very high read/write rates and the scale might bemillions of operations per second.
    - The application may need to examine data changes over time.
- Thus, BigTable can be viewed as a distributed multilevel map. It provides a fault tolerantand persistent database as in a storage service.
- The BigTable system is scalable, which means the system has thousands of servers, terabytes of in-memory data, peta bytes of disk based data, millions of reads/writes per second and efficient scans.

- BigTable is a self managing system (i.e., servers can be added/removed dynamically and it features automatic load balancing).
- Chubby, Google's Distributed Lock Service Chubby is intended to provide a coarse- grained locking service.
- It can store small files inside Chubby storage which provides a simple namespace as afile system tree.
- The files stored in Chubby are quite small compared to the huge files in GFS.

Introduction to Amazon Web Services:

Amazon Web Services (AWS) is a leading top platform in providing the web services of various domains. AWS follows the trends of digital IT and comes up needy services with optimized performances covering a wide range of services from Compute to Storage. It covers a wider range of customers of different domains to expand their business operations. This Article covers the fundamentals of AWS and its scope of IT business.

What Is AWS And Why Is It Used?

AWS stands for Amazon Web Services, It is an expanded **cloud computing platform** provided by Amazon Company. AWS provides a wide range of services with a **pay-as-per-use pricing model** over the Internet such as Storage, Computing power, Databases, Machine Learning services, and much more. AWS facilitates for both businesses and individual users with effectively hosting the applications, storing the data securely, and making use of a wide variety of tools and services improving management flexibility for IT resources.

History Of AWS

Then providing Simple Storage Service (Amazon S3) revolutionized with scalable management of Storage. Coming up with effective compute and storage services and providing them rental basis helped many startup companies and users with the cost of manual Hardware Infrasture setup. Introducing the concept of serverless computing with AWS lambda services enhanced its business globally. It came up with services like Elastic Beanstalk made the deployment of applications much easier bringing large audiences. AWS always came with diverse array of services offering with technical innovations, updated services with current trends. AWS has emerged as a powerhouse in the world of Cloud Computing.

How AWS Works?

AWS comes up with its own network infrastructure on establishing the datacenters in different regions mostly all over the world. Its global Infrastructure acts as a backbone for operations and services provided by AWS. It facilitates the users on creating secure environments using Amazon VPCs (Virtual Private Clouds). Essential services like Amazon EC2 and Amazon S3 for utilizing the compute and storage service with elastic scaling. It supports the dynamic scaling of the applications with the services such as Auto Scaling and Elastic Load Balancing (AWS ELB). It provides a good user-friendly AWS Management Console facilitating seamless configuration and management of AWS services to the Users. Its Architecture ensures high availability , fault tolerance making AWS as a versatile powerful Cloud Computing Platform.

AWS Fundamentals

In the Journey of AWS, understanding the key concepts such as Regions, Availability Zones, Global Network Infrastructure, etc is crucial. The fundamentals of AWS keep on maintaining the applications reliable and scalable with services globally with coming to a strategic

deployment of resources for optimal performance and resilience. The following are the some of the main fundamentals of AWS:

- **Regions:** AWS provide the services with respective division of regions. The regions are divided based on geographical areas/locations and will establish data centers. Based on need and traffic of users, the scale of data centers is depended to facilitate users with low-latencies of servcies.
- **Availability Zones (AZ):** To prevent the Data centers for the Natural Calamities or any other disasters. The Datacenters are established as sub sections with isolated locations to enhance fault tolerance and disaster recovery management.
- **Global Network Infrastructure:** AWS ensures the reliability and scalability of services through setting up its own AWS Network Infrastructure globally. It helps in better management of data transmissions for optimized performance and security reliance.

Top AWS Services

In the rapid revolution of Cloud Computing, AWS facilitates with wide variety of services respect to the fields and needs. The following are the top AWS services that are in wide usage:

- **Amazon EC2(Elastic Compute Cloud) :** It provides the Scalable computing power via cloud allowing the users to run applications and manage the workloads over their remotely.
- **Amazon S3 (Simple Storage Service):** It offers scalable object Storage as a Service with high durability for storing and retrieving any amount of data.
- **AWS Lambda**: It is a service in Serverless Architecture with Function as a Service facilitating serverless computing i.e., running the code on response to the events, the background environment management of servers is handled by aws automatically. It helps the developers to completely focus on the logic of code build.
- **Amazon RDS (Relational Database Service):** This is an aws service that simplifies the management of database providing high available relational databases in the cloud.
- **Amazon VPC (Virtual Private Cloud):** It enables the users to create isolated networks with option of public and private expose within the AWS cloud, providing safe and adaptable configurations of their resources.

To know more about refer the Article – Top 25 AWS Service List

Advantages Of Amazon Web Services

- AWS allows you to easily scale your resources up or down as your needs change, helping you to save money and ensure that your application always has the resources it needs.
- AWS provides a highly reliable and secure infrastructure, with multiple data centers and a commitment to 99.99% availability for many of its services.
- AWS offers a wide range of services and tools that can be easily combined to build and deploy a variety of applications, making it highly flexible.
- AWS offers a pay-as-you-go pricing model, allowing you to only pay for the resources you actually use and avoid upfront costs and long-term commitments.

Disadvantages Of Amazon Web Services

- AWS can be complex, with a wide range of services and features that may be difficult to understand and use, especially for new users.
- AWS can be expensive, especially if you have a high-traffic application or need to run multiple services. Additionally, the cost of services can increase over time, so you need to regularly monitor your spending.

- While AWS provides many security features and tools, securing your resources on AWS can still be challenging, and you may need to implement additional security measures to meet your specific requirements.
- AWS manages many aspects of the infrastructure, which can limit your control over certain parts of your application and environment.

Applications Of AWS

The AWS services are using by both startup and MNC companies as per their usecase. The startup companies are using overcome hardware infrasture cost and applications deployments effectively with cost and performance. Whereas large scale companies are using AWS cloud services for the management of their Infrastructure to completely focus on the development of products widely. The following the Real-world industrial use-cases of AWS services:

- **Netflix:** The Large streaming gaint using AWS for the storage and scaing of the applications for ensuring seamless content delivery with low latency without interruptions to millions of users globally.
- **Airbnb:** By utilizing AWS, Airbnb manages the various workloads and provides insurable and expandable infrastructure for its virtual marketplace and lodging offerings.
- **NASA's Jet Propulsion Laboratory:** It takes the help of AWS services to handle and analyze large-scale volumes of data related to vital scientific research missions and space exploration.
- **Capital One:** A financial Company that is utilizing AWS for its security and compliance while delivering innovative banking services to its customers.

AWS Global Infrastructure

The AWS global infrastructure is massive and is divided into geographical regions. The geographical regions are then divided into separate availability zones. While selecting the geographical regions for AWS, three factors come into play

- Optimizing Latency
- Reducing cost
- Government regulations (Some services are not available for some regions)

Each region is divided into at least two availability zones that are physically isolated from each other, which provides business continuity for the infrastructure as in a distributed system. If one zone fails to function, the infrastructure in other availability zones remains operational. The largest region North Virginia (US-East), has six availability zones. These availability zones are connected by high-speed fiber-optic networking.

There are over 100 edge locations distributed all over the globe that are used for the CloudFront (content delivery network). CloudFront can cache frequently used content such as images and videos(live streaming videos also) at edge locations and distribute it to edge locations across the globe for high-speed delivery and low latency for end-users. It also protects from DDOS attacks.

AWS Management Console

The AWS management console is a web-based interface to access AWS. It requires an AWS account and also has a smartphone application for the same purpose. So When you sign in for first time, you see the console home page where you see all the services provided by AWS. Cost monitoring is also done through the console.

AWS resources can also be accessed through various Software Development Kits (SDKs), which allows the developers to create applications as AWS as its backend. There are SDKs for all the major languages(e.g., JavaScript, Python, Node.js, .Net, PHP, Ruby, Go, C++). There are mobile SDKs for Android, iOS, React Native, Unity, and Xamarin. AWS can also be accessed by making HTTP calls using the AWS-API. AWS also provides a AWS Command Line Interface (CLI) for remotely accessing the AWS and can implement scripts

to automate many processes. This Console is also available as an app for Android and iOS. For mobile apps, you can simply download AWS console app.

AWS Cloud Computing Models

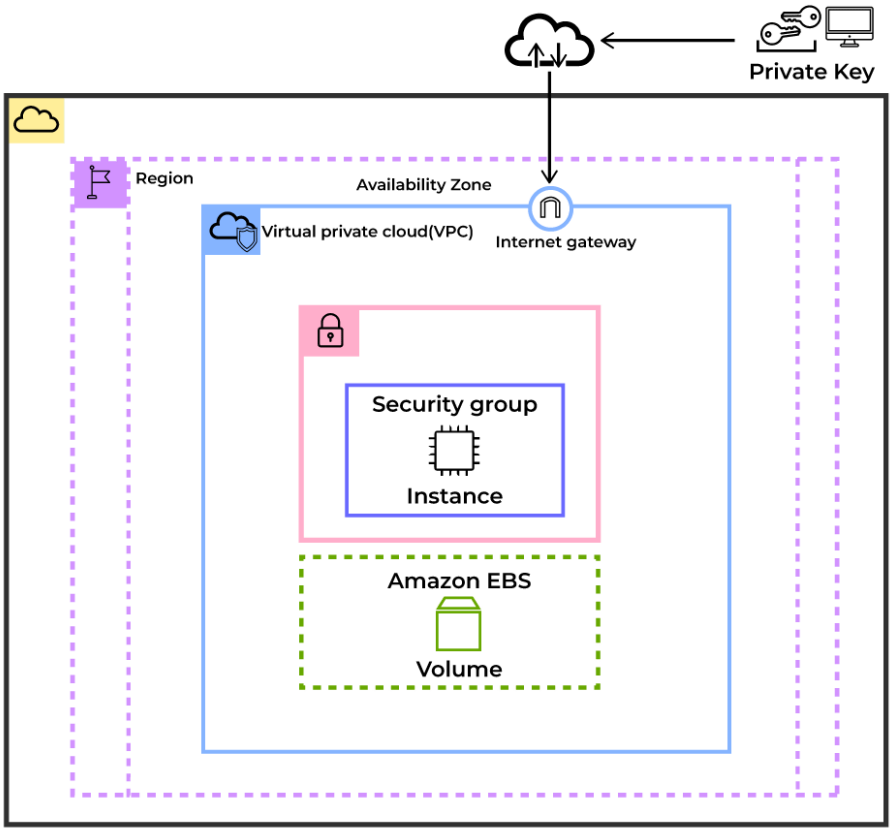
There are three cloud computing models available on AWS.

1. **Infrastructure as a Service (IaaS):** It is the basic building block of cloud IT. It generally provides access to data storage space, networking features, and computer hardware(virtual or dedicated hardware). It is highly flexible and gives management controls over the IT resources to the developer. For example, VPC, EC2, EBS.
2. **Platform as a Service (PaaS):** This is a type of service where AWS manages the underlying infrastructure (usually operating system and hardware). This helps the developer to be more efficient as they do not have to worry about undifferentiated heavy lifting required for running the applications such as capacity planning, software maintenance, resource procurement, patching, etc., and focus more on deployment and management of the applications. For example, RDS, EMR, ElasticSearch.
3. **Software as a Service(SaaS):** It is a complete product that usually runs on a browser. It primarily refers to end-user applications. It is run and managed by the service provider. The end-user only has to worry about the application of the software suitable to its needs. For example, Saleforce.com, Web-based email, Office 365.

Compute with EC2:

EC2 stands for Elastic Compute Cloud. EC2 is an on-demand computing service on the AWS cloud platform. Under computing, it includes all the services a computing device can offer to you along with the flexibility of a virtual environment. It also allows the user to configure their instances as per their requirements i.e. allocate the RAM, ROM, and storage according to the need of the current task. Even the user can dismantle the virtual device once its task is completed and it is no more required. For providing, all these scalable resources AWS charges some bill amount at the end of every month, the bill amount is entirely dependent on your usage. EC2 allows you to rent virtual computers. The provision of servers on AWS Cloud is one of the easiest ways in EC2. EC2 has resizable capacity. EC2 offers security, reliability, high performance, and cost-effective infrastructure so as to meet the demanding business needs.

Amazon Web Service EC2 is a web service which is provided by the AWS cloud which is secure, resizable, and scalable. These virtual machines are pre-configured with the operating systems and some of the required software. Instead of managing the infrastructure AWS will do that so you can just launch and terminate the EC2 instance whenever you want. You can scale up and down the EC2 instance depending on the incoming traffic. The other advantage of AWS EC2 is that you need to pay only for how much you use it is like the pay-as-you-go model.



Use Cases of Amazon EC2 (Elastic Compute Cloud)

The following are the use cases of Amazon EC2:

1. **Deploying Application:** In the AWS EC2 instance, you can deploy your application like **.jar,.war,** or **.ear** application without maintaining the underlying infrastructure.
2. **Scaling Application:** Once you deployed your web application in the EC2 instance know you can scale your application based upon the demand you are having by scaling the AWS EC2-Instance.
3. **Deploying The ML Models:** You can train and deploy your ML models in the EC2-instance because it offers up to 400 Gbps), and storage services purpose-built to optimize the price performance for ML projects.
4. **Hybrid Cloud Environment:** You can deploy your web application in EC2-Instance and you can connect to the database which is deployed in the on-premises servers.
5. **Cost-Effective:** Amazon EC2-instance is cost-effective so you can deploy your gaming application in the Amazon EC2-Instances

AWS EC2 Instance Types

Different Amazon EC2 instance types are designed for certain activities. Consider the unique requirements of your workloads and applications when choosing an instance type. This might include needs for computing, memory, or storage.

The AWS EC2 Instance types are as follows:

- General Purpose Instances
- Compute Optimized Instances
- Memory-Optimized Instances
- Storage Optimized Instances
- Accelerated Computing Instances

1. General Purpose Instances

- It provides the balanced resources for a wide range of workloads.
- It is suitable for web servers, development environments, and small databases.

Examples: T3, M5 instances.

2. Compute Optimized Instances

- It provides high-performance processors for compute-intensive applications.
- It will be Ideal for high-performance web servers, scientific modeling, and batch processing.

Examples: C5, C6g instances.

3. Memory-Optimized Instances

- High memory-to-CPU ratios for large data sets.
- Perfect for in-memory databases, real-time big data analytics, and high-performance computing (HPC).

Examples: R5, X1e instances.

4. Storage Optimized Instances

- It provides optimized resource of instance for high, sequential read and write access to large data sets.
- Best for data warehousing, Hadoop, and distributed file systems.

Examples: I3, D2 instances.

5. Accelerated Computing Instances

- It facilitates with providing hardware accelerators or co-processors for graphics processing and parallel computations.
- It is ideal for machine learning, gaming, and 3D rendering.

Examples: P3, G4 instances.

Features of AWS EC2 (Elastic Compute Cloud)

The following are the features of AWS EC2:

1. AWS EC2 Functionality

EC2 provides its users with a true virtual computing platform, where they can use various operations and even launch another EC2 instance from this virtually created environment. This will increase the security of the virtual devices. Not only creating but also EC2 allows us to customize our environment as per our requirements, at any point of time during the life span of the virtual machine. Amazon EC2 itself comes with a set of default AMI(Amazon Machine Image) options supporting various operating systems along with some pre-configured resources like RAM, ROM, storage, etc. Besides these AMI options, we can also create an AMI curated with a combination of default and user-defined configurations. And for future purposes, we can store this user-defined AMI, so that next time, the user won't have to re-configure a new AMI(Amazon Machine Image) from scratch. Rather than this whole process, the user can simply use the older reference while creating a new EC2 machine.

2. AWS EC2 Operating Systems

Amazon EC2 includes a wide range of operating systems to choose from while selecting your AMI. Not only are these selected options, but users are also even given the privilege to upload their own operating systems and opt for that while selecting AMI during launching an EC2 instance. Currently, AWS has the following most preferred set of operating systems available on the EC2 console.



- Amazon Linux
- Windows Server
- Ubuntu Server
- SUSE Linux
- Red Hat Linux

3. AWS EC2 Software

Amazon is single-handedly ruling the cloud computing market, because of the variety of options available on EC2 for its users. It allows its users to choose from various software present to run on their EC2 machines. This whole service is allocated to AWS Marketplace on the AWS platform. Numerous software like SAP, LAMP, Drupal, etc are available on AWS to use.

4. AWS EC2 Scalability and Reliability

EC2 provides us the facility to scale up or scale down as per the needs. All dynamic scenarios can be easily tackled by EC2 with the help of this feature. And because of the flexibility of volumes and snapshots, it is highly reliable for its users. Due to the scalable nature of the machine, many organizations like Flipkart, and Amazon rely on these days whenever humongous traffic occurs on their portals.

Pricing of AWS EC2 (Elastic Compute Cloud) Instance

The pricing of AWS EC2-instance is mainly going to depend upon the type of instance you are going to choose. The following are the pricing charges on some of the EC2-Instances.

1. **On-Demand Instances:** The On-Demand instance is like a pay-as-you-go model where you have to pay only for the time you are going to use if the instance is stopped then the billing for that instance will be stopped when it was in the running state then you are going to be charged. The billing will be done based on the time EC2-Instance is running.
2. **Reserved Instances:** Reversed Instance is like you are going to give the commitment to the AWS by buying the instance for one year or more than one year by the requirement to your organization. Because you are giving one year of Commitment to the AWS they will discount the price on that instance.
3. **Spot Instances:** You have to bid the instances and who will win the bid they are going to get the instance for use but you can't save the data which is used in this type of instance.

SQL Database with AWS RDS:

Amazon Relational Database Service (Amazon RDS):

Amazon Relational Database Service (Amazon RDS) is a web service that makes it easier to set up, operate, and scale a relational database in the AWS Cloud. It provides cost-efficient, resizable capacity for an industry-standard relational database and manages common database

administration tasks.
Amazon RDS is responsible for hosting the software components and infrastructure of DB instances and DB clusters. You are responsible for query tuning, which is the process of adjusting SQL queries to improve performance. Query performance is highly dependent on database design, data size, data distribution, application workload, and query patterns, which can vary greatly. Monitoring and tuning are highly individualized processes that you own for your RDS databases. You can use Amazon RDS Performance Insights and other tools to identify problematic queries.

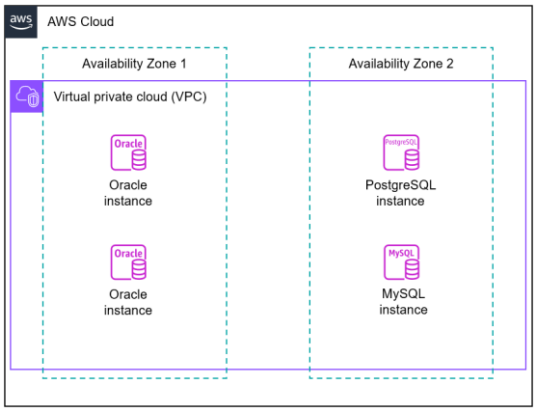
Advantages of Amazon RDS

Amazon RDS is a managed database service. It's responsible for most management tasks. By eliminating tedious manual processes, Amazon RDS frees you to focus on your application and your users.
Amazon RDS provides the following principal advantages over database deployments that aren't fully managed:

- You can use database engines that you are already familiar with: IBM Db2, MariaDB, Microsoft SQL Server, MySQL, Oracle Database, and PostgreSQL.
- Amazon RDS manages backups, software patching, automatic failure detection, and recovery.
- You can turn on automated backups, or manually create your own backup snapshots. You can use these backups to restore a database. The Amazon RDS restore process works reliably and efficiently.
- You can get high availability with a primary DB instance and a synchronous secondary DB instance that you can fail over to when problems occur. You can also use read replicas to increase read scaling.
- In addition to the security in your database package, you can control access by using AWS Identity and Access Management (IAM) to define users and permissions. You can also help protect your databases by putting them in a virtual private cloud (VPC).

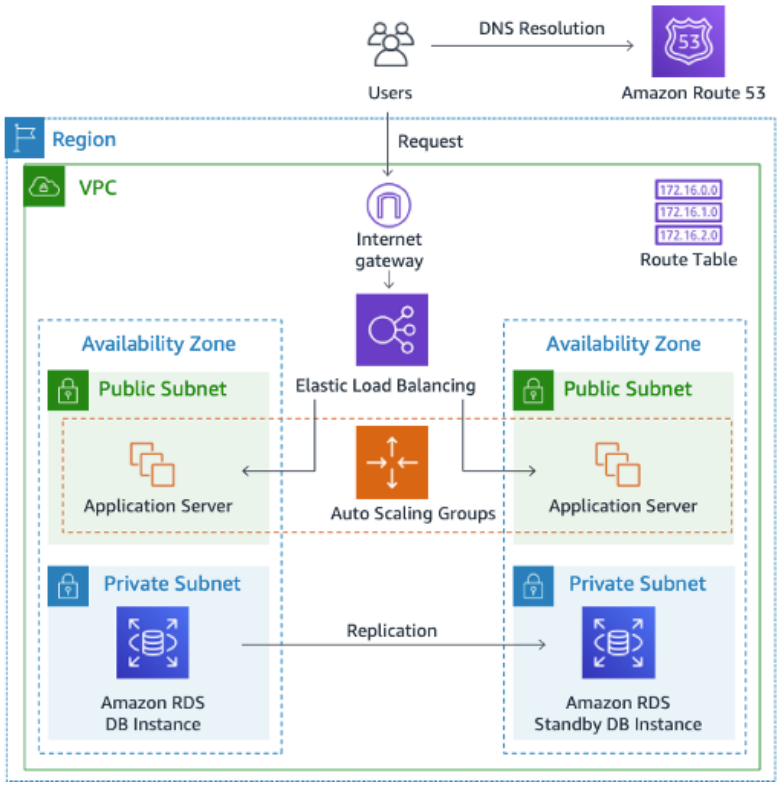
Amazon RDS DB instances

A *DB instance* is an isolated database environment in the AWS Cloud. The basic building block of Amazon RDS is the DB instance. Your DB instance can contain one or more user-created databases. The following diagram shows a virtual private cloud (VPC) that contains two Availability Zones, with each AZ containing two DB instances.



Amazon RDS application architecture: example

The following image shows a typical use case of a dynamic website that uses Amazon RDS DB instances for database storage:



The primary components of the preceding architecture are as follows:

Elastic Load Balancing

AWS routes user traffic through Elastic Load Balancing. A load balancer distributes workloads across multiple compute resources, such as virtual servers. In this sample use case, the Elastic Load Balancer forwards client requests to application servers.

Application servers

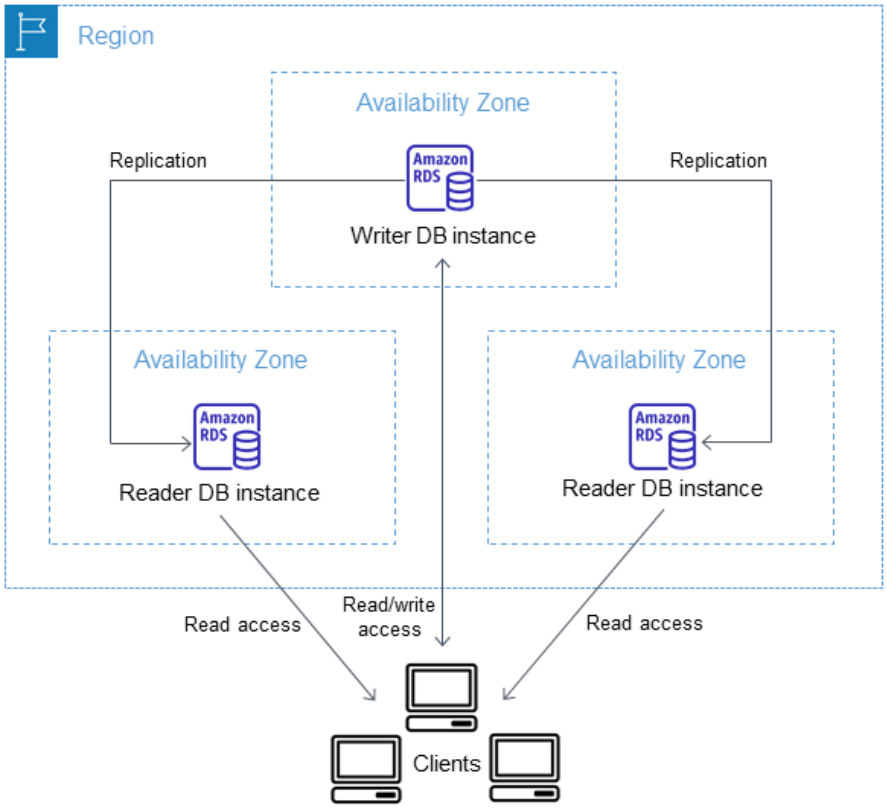
Application servers interact with RDS DB instances. An application server in AWS is typically hosted on EC2 instances, which provide scalable computing capacity. The application servers reside in public subnets with different Availability Zones (AZs) within the same Virtual Private Cloud (VPC). .

RDS DB instances

The EC2 application servers interact with RDS DB instances. The DB instances reside in private subnets within different Availability Zones (AZs) within the same Virtual Private Cloud (VPC). Because the subnets are private, no requests from the internet are permitted.

The primary DB instance replicates to another DB instance, called a *read replica*. Both DB instances are in private subnets within the VPC, which means that Internet

users can't access them directly.



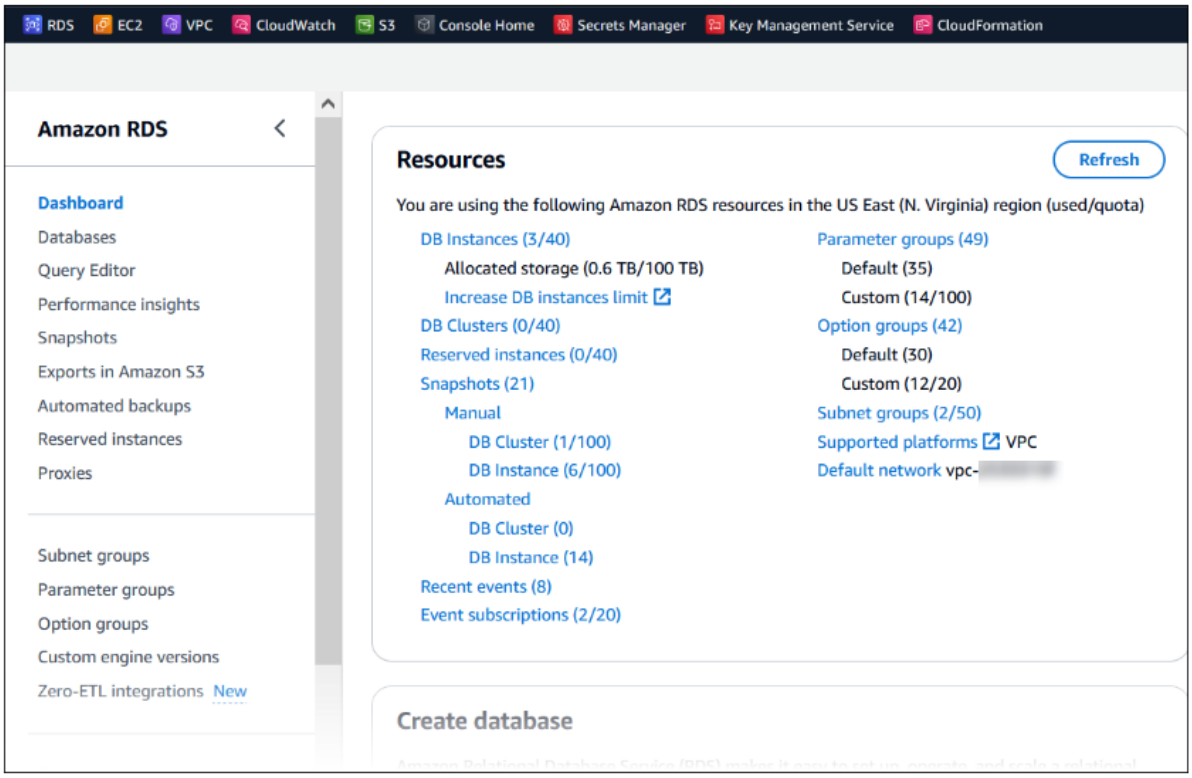
User interfaces to Amazon RDS

You can interact with Amazon RDS in multiple ways:

- AWS Management Console
- Command line interface
- Amazon RDS APIs

AWS Management Console

The AWS Management Console is a simple web-based user interface. You can manage your DB instances from the console with no programming required. To access the Amazon RDS console, sign in to the AWS Management Console and open the Amazon RDS console at https://console.aws.amazon.com/rds/



Command line interface

You can use the AWS Command Line Interface (AWS CLI) to access the Amazon RDS API interactively. To install the AWS CLI, see Installing the AWS Command Line Interface. To begin using the AWS CLI for RDS, see AWS Command Line Interface reference for Amazon RDS.

Amazon RDS APIs

If you are a developer, you can access the Amazon RDS programmatically using APIs. For more information, see Amazon RDS API reference.

For application development, we recommend that you use one of the AWS Software Development Kits (SDKs). The AWS SDKs handle low-level details such as authentication, retry logic, and error handling, so that you can focus on your application logic. AWS SDKs are available for a wide variety of languages.

To create an Amazon RDS for SQL DB instance, use the Amazon RDS management tools or interfaces. You can then do the following:

- Resize your DB instance
- Authorize connections to your DB instance
- Create and restore from backups or snapshots
- Create Multi-AZ secondaries
- Create read replicas
- Monitor the performance of your DB instance

NoSQL with Document DB:

Amazon DocumentDB (with MongoDB compatibility) is a fast, reliable, and fully managed database service. Amazon DocumentDB makes it easy to set up, operate, and scale MongoDB-

compatible databases in the cloud. With Amazon DocumentDB, you can run the same application code and use the same drivers and tools that you use with MongoDB.



The following are some high-level features of Amazon DocumentDB:

- Amazon DocumentDB supports two types of clusters: instance-based clusters and elastic clusters. Elastic clusters support workloads with millions of reads/writes per second and petabytes of storage capacity. For more information about elastic clusters, see Using Amazon DocumentDB elastic clusters. The content below refers to Amazon DocumentDB instance-based clusters .
- Amazon DocumentDB automatically grows the size of your storage volume as your database storage needs grow. Your storage volume grows in increments of 10 GB, up to a maximum of 128 TiB. You don't need to provision any excess storage for your cluster to handle future growth.
- With Amazon DocumentDB, you can increase read throughput to support high-volume application requests by creating up to 15 replica instances. Amazon DocumentDB replicas share the same underlying storage, lowering costs and avoiding the need to perform writes at the replica nodes. This capability frees up more processing power to serve read requests and reduces the replica lag time—often down to single digit milliseconds. You can add replicas in minutes regardless of the storage volume size. Amazon DocumentDB also provides a reader endpoint, so the application can connect without having to track replicas as they are added and removed.
- Amazon DocumentDB lets you scale the compute and memory resources for each of your instances up or down. Compute scaling operations typically complete in a few minutes.

- Amazon DocumentDB runs in Amazon Virtual Private Cloud (Amazon VPC), so you can isolate your database in your own virtual network. You can also configure firewall settings to control network access to your cluster.
- Amazon DocumentDB continuously monitors the health of your cluster. On an instance failure, Amazon DocumentDB automatically restarts the instance and associated processes. Amazon DocumentDB doesn't require a crash recovery replay of database redo logs, which greatly reduces restart times. Amazon DocumentDB also isolates the database cache from the database process, enabling the cache to survive an instance restart.
- On instance failure, Amazon DocumentDB automates failover to one of up to 15 Amazon DocumentDB replicas that you create in other Availability Zones. If no replicas have been provisioned and a failure occurs, Amazon DocumentDB tries to create a new Amazon DocumentDB instance automatically.
- The backup capability in Amazon DocumentDB enables point-in-time recovery for your cluster. This feature allows you to restore your cluster to any second during your retention period, up to the last 5 minutes. You can configure your automatic backup retention period up to 35 days. Automated backups are stored in Amazon Simple Storage Service (Amazon S3), which is designed for 99.999999999% durability. Amazon DocumentDB backups are automatic, incremental, and continuous, and they have no impact on your cluster performance.
- With Amazon DocumentDB, you can encrypt your databases using keys that you create and control through AWS Key Management Service (AWS KMS). On a database cluster running with Amazon DocumentDB encryption, data stored at rest in the underlying storage is encrypted. The automated backups, snapshots, and replicas in the same cluster are also encrypted.
- Amazon DocumentDB is authorized under Federal Risk and Authorization Management Program (FedRAMP). It has FedRAMP High authorization for AWS GovCloud (US) regions and FedRAMP Moderate authorization for AWS US East/West Regions.

Cloud Storage Providers: S3 – Storage types in S3:

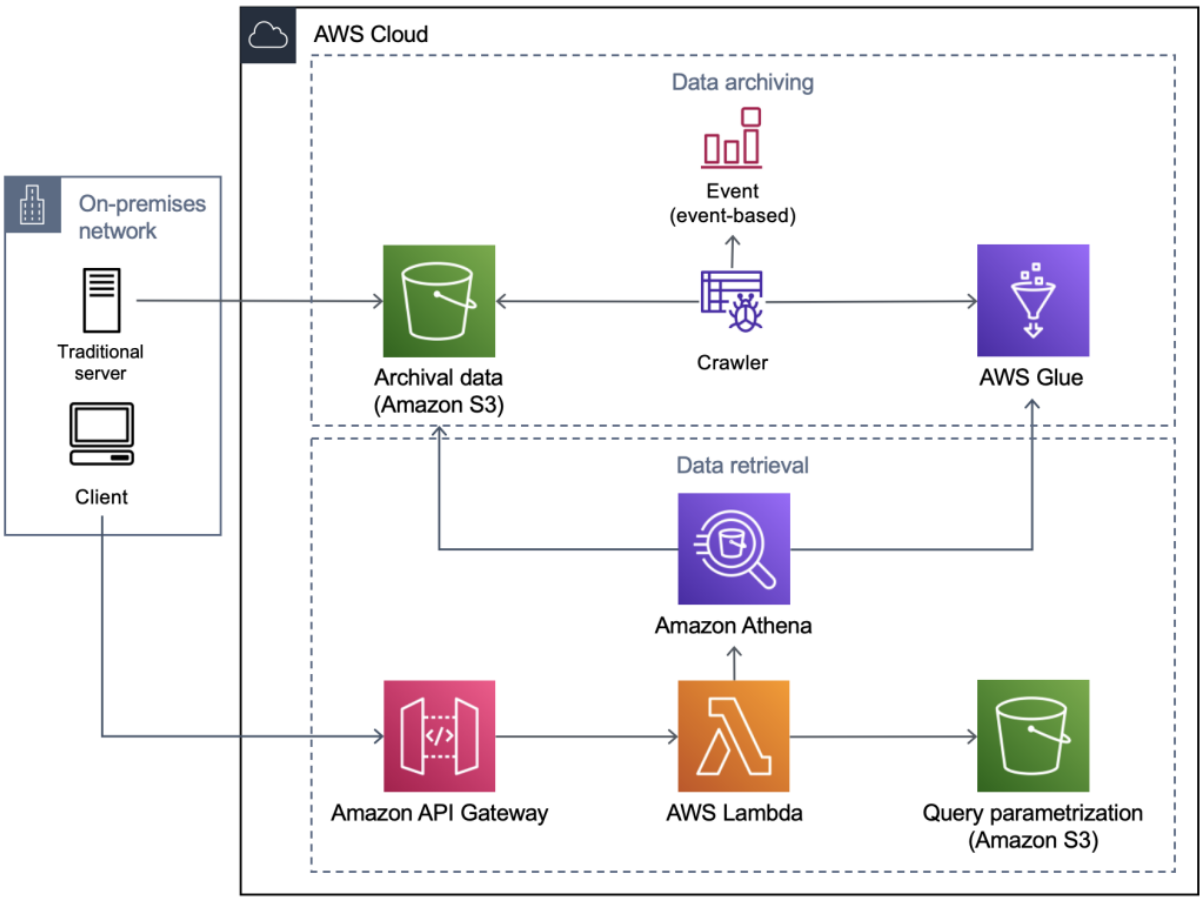
Amazon Simple Storage Service (Amazon S3) is an object storage service that offers industry-leading scalability, data availability, security, and performance. Customers of all sizes and industries can use Amazon S3 to store and protect any amount of data for a range of use cases, such as data lakes, websites, mobile applications, backup and restore, archive, enterprise applications, IoT devices, and big data analytics. Amazon S3 provides management features so that you can optimize, organize, and configure access to your data to meet your specific business, organizational, and compliance requirements.

Amazon S3 is an object storage service that stores data as objects within buckets. An *object* is a file and any metadata that describes the file. A *bucket* is a container for objects.

To store your data in Amazon S3, you first create a bucket and specify a bucket name and AWS Region. Then, you upload your data to that bucket as objects in Amazon S3. Each object has a *key* (or *key name*), which is the unique identifier for the object within the bucket.

S3 provides features that you can configure to support your specific use case. For example, you can use S3 Versioning to keep multiple versions of an object in the same bucket, which allows you to restore objects that are accidentally deleted or overwritten.

Buckets and the objects in them are private and can be accessed only if you explicitly grant access permissions. You can use bucket policies, AWS Identity and Access Management (IAM) policies, access control lists (ACLs), and S3 Access Points to manage access.



What is Amazon S3 Used for?

Amazon S3 is used for various purposes in the Cloud because of its robust features with scaling and Securing of data. It helps people with all kinds of use cases from fields such as Mobile/Web applications, Big data, Machine Learning and many more. The following are a few Wide Usage of Amazon S3 service.

- **Data Storage:** Amazon s3 acts as the best option for scaling both small and large storage applications. It helps in storing and retrieving the data-intensitive applications as per needs in ideal time.

- **Backup and Recovery:** Many Organizations are using Amazon S3 to backup their critical data and maintain the data durability and availability for recovery needs.
- **Hosting Static Websites:** Amazon S3 facilitates in storing HTML, CSS and other web content from Users/developers allowing them for hosting Static Websites benefiting with low-latency access and cost-effectiveness. To know more detailing refer this Article – How to host static websites using Amazon S3
- **Data Archiving:** Amazon S3 Glacier service integration helps as a cost-effective solution for long-term data storing which are less frequently accessed applications.
- **Big Data Analytics:** Amazon S3 is often considered as data lake because of its capacity to store large amounts of both structured and unstructured data offering seamless integration with other AWS Analytics and AWS Machine Learning Services.

What is an Amazon S3 bucket?

Amazon S3 bucket is a fundamental Storage Container feature in AWS S3 Service. It provides a secure and scalable repository for storing of Objects such as Text data, Images, Audio and Video files over AWS Cloud. Each S3 bucket name should be named globally unique and should be configured with ACL (Access Control List).

How Does Amazon S3 works?

Amazon S3 works on organizing the data into unique S3 Buckets, customizing the buckets with Acccess controls. It allows the users to store objects inside the S3 buckets with facilitating features like versioning and lifecycle management of data storage with scaling. The following are a few main features of Amazon s3:

1. Amazon S3 Buckets and Objects

Amazon S3 Bucket: Data, in S3, is stored in containers called *buckets*. Each bucket will have its own set of policies and configurations. This enables users to have more control over their data. Bucket Names must be unique. Can be thought of as a parent folder of data. There is a limit of 100 buckets per AWS account. But it can be increased if requested by AWS support.

Amazon S3 Objects: Fundamental entity type stored in AWS S3.You can store as many objects as you want to store. The maximum size of an AWS S3 bucket is 5TB. It consists of the following:

- Key
- Version ID
- Value
- Metadata
- Subresources
- Access control information
- Tags

2. Amazon S3 Versioning and Access Control

S3 Versioning: Versioning means always keeping a record of previously uploaded files in S3. Points to Versioning are not enabled by default. Once enabled, it is enabled for all objects in a bucket. Versioning keeps all the copies of your file, so, it adds cost for storing multiple copies of your data. For example, 10 copies of a file of size 1GB will have you charged for using 10GBs for S3 space. Versioning is helpful to prevent unintended overwrites and deletions. Objects with the same key can be stored in a bucket if versioning is enabled (since they have a unique version ID). To know more about versioning refer this article – Amazon S3 Versioning

Access control lists (ACLs): A document for verifying access to S3 buckets from outside your AWS account. An ACL is specific to each bucket. You can utilize S3 Object Ownership, an Amazon S3 bucket-level feature, to manage who owns the objects you upload to your bucket and to enable or disable ACLs.

3. Bucket policies and Life Cycles

Bucket Policies: A document for verifying the access to S3 buckets from within your AWS account, controls which services and users have what kind of access to your S3 bucket. Each bucket has its own Bucket Policies.

Lifecycle Rules: This is a cost-saving practice that can move your files to AWS Glacier (The AWS Data Archive Service) or to some other S3 storage class for cheaper storage of old data or completely delete the data after the specified time. To know more about refer this article – Amazon S3 Life Cycle Management

4. Keys and Null Objects

Keys: The key, in S3, is a unique identifier for an object in a bucket. For example in a bucket 'ABC' your GFG.java file is stored at javaPrograms/GFG.java then 'javaPrograms/GFG.java' is your object key for GFG.java.

Null Object: Version ID for objects in a bucket where versioning is suspended is null. Such objects may be referred to as null objects.List) and Other settings for managing data efficiently.

How To Use an Amazon S3 Bucket?

You can use the Amazon S3 buckets by following the simple steps which are mentioned below. To know more how to configure about Amazon S3 refer to the Amazon S3 – Creating a S3 Bucket.

Step 1: Login into the Amazon account with your credentials and search form S3 and click on the S3. Now click on the option which is **"Create bucket"** and configure all the options which are shown while configuring.

Step 2: After configuring the AWS bucket now upload the objects into the buckets based upon your requirement. By using the AWS console or by using AWS CLI following is the command to upload the object into the AWS S3 bucket.

aws s3 cp <local-file-path> s3://<bucket-name>/

Step 3: You can control the permissions of the objects which was uploaded into the S3 buckets and also who can access the bucket. You can make the bucket public or private by default the S3 buckets will be in private mode.

Step 4: You can manage the S3 bucket lifecycle management by transitioning. Based upon the rules that you defined S3 bucket will be transitioning into different storage classes based on the age of the object which is uploaded into the S3 bucket.

Step 5: You need to turn to enable the services to monitor and analyze S3. You need to enable the S3 access logging to record who was requesting the objects which are in the S3 buckets.

What are the types of S3 Storage Classes?

AWS S3 provides multiple storage types that offer different performance and features and different cost structures.

- **Standard:** Suitable for frequently accessed data, that needs to be highly available and durable.
- **Standard Infrequent Access (Standard IA):** This is a cheaper data-storage class and as the name suggests, this class is best suited for storing infrequently accessed data like log files or data archives. Note that there may be a per GB data retrieval fee associated with the Standard IA class.
- **Intelligent Tiering:** This service class classifies your files automatically into frequently accessed and infrequently accessed and stores the infrequently accessed data in infrequent access storage to save costs. This is useful for unpredictable data access to an S3 bucket.
- **One Zone Infrequent Access (One Zone IA):** All the files on your S3 have their copies stored in a minimum of 3 Availability Zones. One Zone IA stores this data in a

single availability zone. It is only recommended to use this storage class for infrequently accessed, non-essential data. There may be a per GB cost for data retrieval.

- **Reduced Redundancy Storage (RRS):** All the other S3 classes ensure the durability of 99.999999999%. RRS only ensures 99.99% durability. AWS no longer recommends RRS due to its less durability. However, it can be used to store non-essential data.

To know more about , refer this article – Amazon S3 Storage Classes

How to Access Amazon S3 Bucket?

You can work and access the Amazon S3 bucket by using any one of the following methods

1. AWS Management Console
2. AWS CLI Commands
3. Programming Scripts (Using boto3 library of Python)

1. AWS Management Console

You can access the AWS S3 bucket using the AWS management console which is a web-based user interface. Firstly you need to create an AWS account and login to the Web console and from there you can choose the S3 bucket option from Amazon S3 service. (AWS Console >> Amazon S3 >> S3 Buckets)

2. AWS CLI Commands

In this methods firstly you have to install the aws cli software in the terminal and try on configuring the aws account with access key, secret key and the default region. Then on taking the **`aws –help` ,** you can figure out the s3 service usage. For example , To view try on running following command:

```
aws s3 ls
```

3. Programming scripts

You can configure the Amazon S3 bucket by using a scripting programing languages like Python and with using libraries such as **boto3 library** you can perform the AWS S3 tasks. To know more about refer this article – How to access Amazon S3 using python script.

AWS S3 Bucket Permissions

You can manage the permission of S3 buckets by using several methods following are a few of them.

1. **Bucket Policies:** Bucket policies can be attached directly to the S3 bucket and they are in JSON format which can perform the bucket level operations. With the help of bucket policies, you can grant permissions to the users who can access the objects present in the bucket. If you grant permissions to any user he can download, and upload the objects to the bucket. You can create the bucket policy by using Python.
2. **Access Control Lists (ACLs):** ACLs are legacy access control mechanisms for S3 buckets instead of ACLs we are using the bucket policies to control the permissions of the S3 bucket. By using ACL you can grant the read, and access to the S3 bucket or you can make the objects public based on the requirements.
3. **IAM Policies:** IAM policies are mostly used to manage the permissions to the users and groups and resources available in the AWS by using the IAM roles options. You can attach an IAM policy to an IAM entity (user, group, or role) granting them access to specific S3 buckets and operations.

The most effective way to control the permissions to the S3 buckets is by using bucket policies.

Features of Amazon S3

- **Durability:** AWS claims Amazon S3 to have a 99.999999999% of durability (11 9's). This means the possibility of losing your data stored on S3 is one in a billion.
- **Availability:** AWS ensures that the up-time of AWS S3 is 99.99% for standard access.

    - Note that availability is related to being able to access data and durability is related to losing data altogether.
- **Server-Side-Encryption (SSE):** AWS S3 supports three types of SSE models:
    - **SSE-S3:** AWS S3 manages encryption keys.
    - **SSE-C:** The customer manages encryption keys.
    - **SSE-KMS:** The AWS Key Management Service (KMS) manages the encryption keys.
- **File Size support:** AWS S3 can hold files of size ranging from 0 bytes to 5 terabytes. A 5TB limit on file size should not be a blocker for most of the applications in the world.
- **Infinite storage space:** Theoretically AWS S3 is supposed to have infinite storage space. This makes S3 infinitely scalable for all kinds of use cases.
- **Pay as you use:** The users are charged according to the S3 storage they hold.

Advantages of Amazon S3

1. **Scalability:** Amazon S3 can be scalable horizontally which makes it handle a large amount of data. It can be scaled automatically without human intervention.
2. **High availability:** AmazonS3 bucket is famous for its high availability nature you can access the data whenever you required it from any region. It offers a Service Level Agreement (SLA) guaranteeing 99.9% uptime.
3. **Data Lifecycle Management:** You can manage the data which is stored in the S3 bucket by automating the transition and expiration of objects based on predefined rules. You can automatically move the data to the Standard-IA or Glacier, after a specified period.
4. **Integration with Other AWS Services:** You can integrate the S3 bucket service with different services in the AWS like you can integrate with the AWS lambda function where the lambda will be triggered based upon the files or objects added to the S3 bucket.

Triggers with Lambda:

Lambda runs your code on a high-availability compute infrastructure and performs all of the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, and logging. With Lambda, all you need to do is supply your code in one of the language runtimes that Lambda supports.

When to use Lambda

Lambda is an ideal compute service for application scenarios that need to scale up rapidly, and scale down to zero when not in demand. For example, you can use Lambda for:

- **File processing:** Use Amazon Simple Storage Service (Amazon S3) to trigger Lambda data processing in real time after an upload.
- **Stream processing:** Use Lambda and Amazon Kinesis to process real-time streaming data for application activity tracking, transaction order processing, clickstream analysis, data cleansing, log filtering, indexing, social media analysis, Internet of Things (IoT) device data telemetry, and metering.

- **Web applications:** Combine Lambda with other AWS services to build powerful web applications that automatically scale up and down and run in a highly available configuration across multiple data centers.
- **IoT backends:** Build serverless backends using Lambda to handle web, mobile, IoT, and third-party API requests.
- **Mobile backends:** Build backends using Lambda and Amazon API Gateway to authenticate and process API requests. Use AWS Amplify to easily integrate with your iOS, Android, Web, and React Native frontends.

When using Lambda, you are responsible only for your code. Lambda manages the compute fleet that offers a balance of memory, CPU, network, and other resources to run your code. Because Lambda manages these resources, you cannot log in to compute instances or customize the operating system on provided runtimes. Lambda performs operational and administrative activities on your behalf, including managing capacity, monitoring, and logging your Lambda functions.

Lambda function invocation methods

After you deploy your Lambda function, you can invoke it in several ways:

- The Lambda console – Use the Lambda console to quickly create a test event to invoke your function.
- The AWS SDK – Use the AWS SDK to programmatically invoke your function.
- The Invoke API – Use the Lambda Invoke API to directly invoke your function.
- The AWS Command Line Interface (AWS CLI) – Use the aws lambda invoke AWS CLI command to directly invoke your function from the command line.
- A function URL HTTP(S) endpoint – Use function URLs to create a dedicated HTTP(S) endpoint that you can use to invoke your function.

All of these methods are *direct* ways to invoke your function. In Lambda, a common use case is to invoke your function based on an event that occurs elsewhere in your application. Some services can invoke a Lambda function with each new event. This is called a trigger. For stream and queue-based services, Lambda invokes the function with batches of records. This is called an event source mapping.

When you invoke a function, you can choose to invoke it synchronously or asynchronously. With synchronous invocation, you wait for the function to process the event and return a response. With asynchronous invocation, Lambda queues the event for processing and returns a response immediately. The InvocationType request parameter in the Invoke API determines

how Lambda invokes your function. A value of RequestResponse indicates synchronous invocation, and a value of Event indicates asynchronous invocation.

To invoke your function over IPv6, use Lambda's public dual-stack endpoints. Dual-stack endpoints support both IPv4 and IPv6. Lambda dual-stack endpoints use the following syntax:

```
protocol://lambda.us-east-1.api.aws
```

You can also use Lambda function URLs to invoke functions over IPv6. Function URL endpoints have the following format:

```
https://url-id.lambda-url.us-east-1.on.aws
```

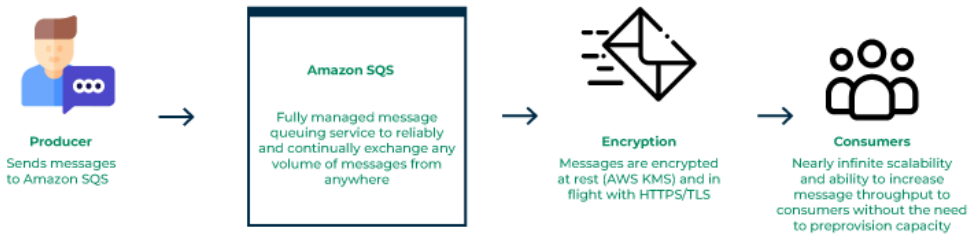
If the function invocation results in an error, for synchronous invocations, view the error message in the response and retry the invocation manually. For asynchronous invocations, Lambda handles retries automatically and can send invocation records to a destination.

Queuing with SQS - Simple Queue Service(SQS):

SQS enables web service applications that help to quickly and reliably queue messages. These messages have one component in their application that generates only when to be consumed by another component. Therefore, the queue is a temporary repository for messages and these messages are awaiting processing. So, Once these messages are processed, the messages also get deleted from the queue. AWS SQS service basically adds messages in a queue and then, Users will pick up these messages from the queue. A queue is a place where you can store your messages until they are extracted from the queue or expired.

How does AWS SQS (Amazon Simple Queue Service) work?

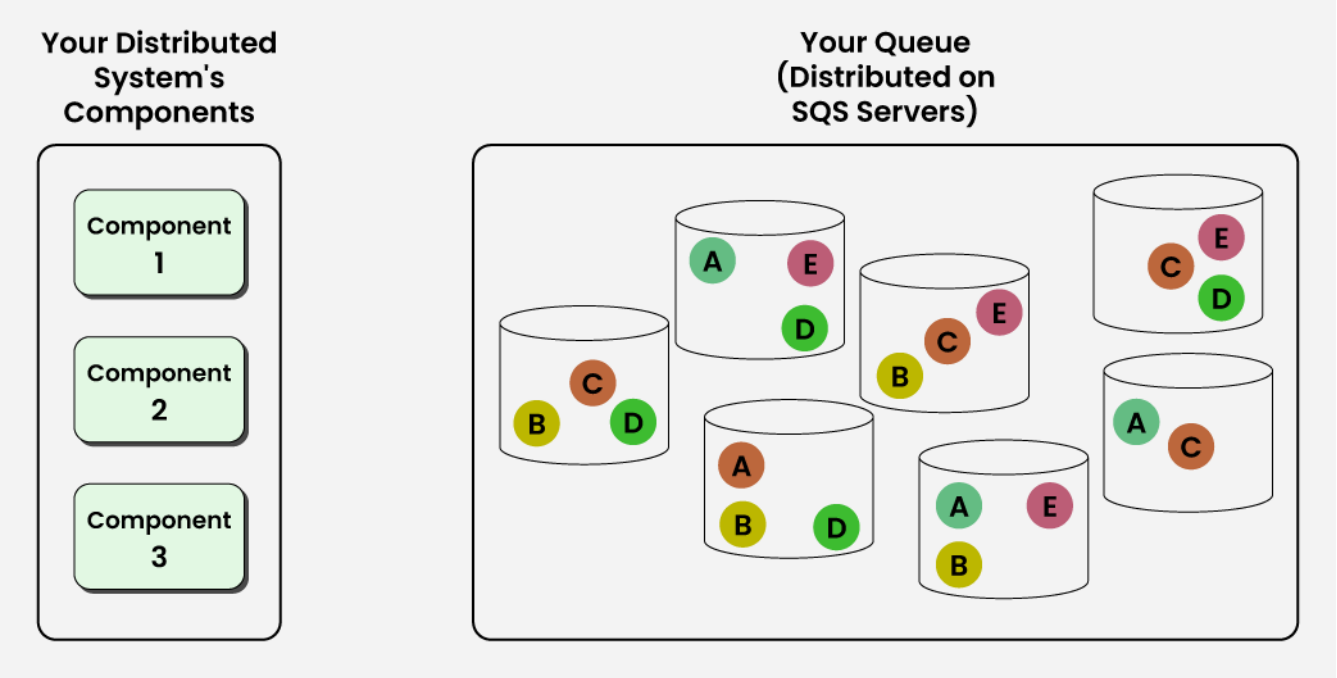
SQS standards for Simple Queue Service offer asynchronous massaged-based communication between two services. Before SQS the communication between the two services was based upon the API calls after the SQS the event producer will notify the consumer in an asynchronous manner.

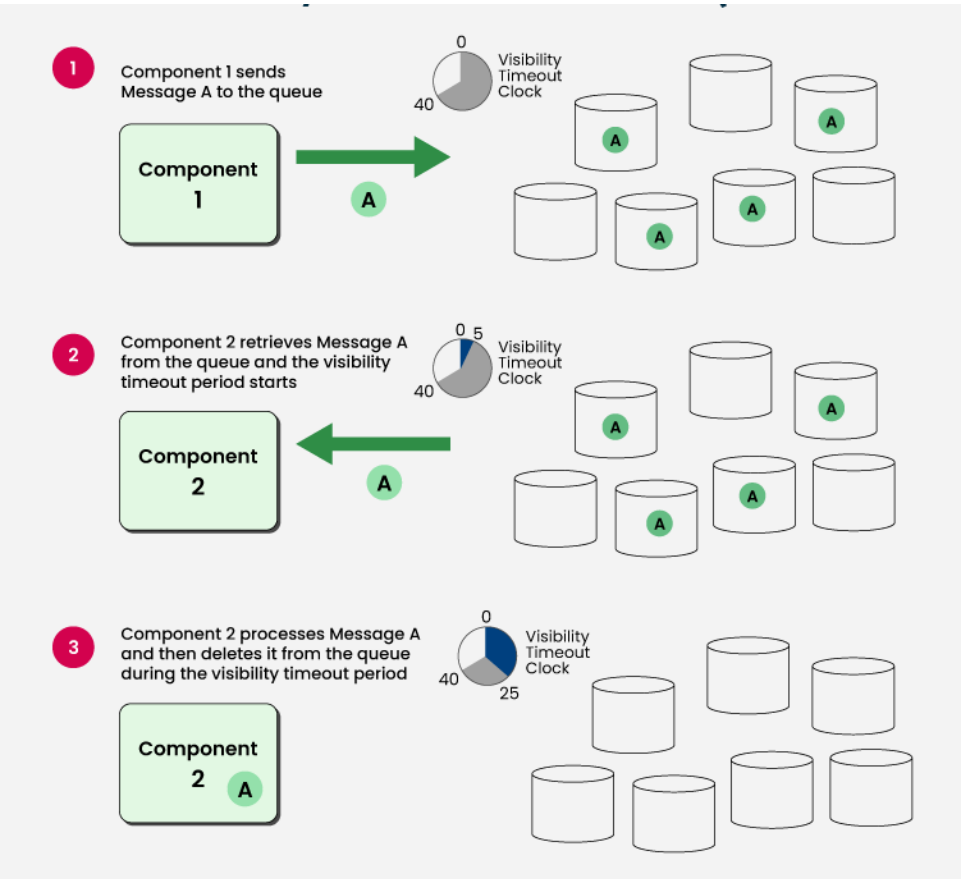


AWS SQS Architecture

In a distributed messaging system, three primary elements are involved: the system components, the queue (which is distributed across Amazon SQS servers), and the messages stored within the queue. In this scenario, your system includes multiple producers (which send messages to the queue) and consumers (which retrieve messages from the

queue). The queue stores messages (such as A, B, C, D, and E) across multiple Amazon SQS servers, ensuring redundancy and high availability.

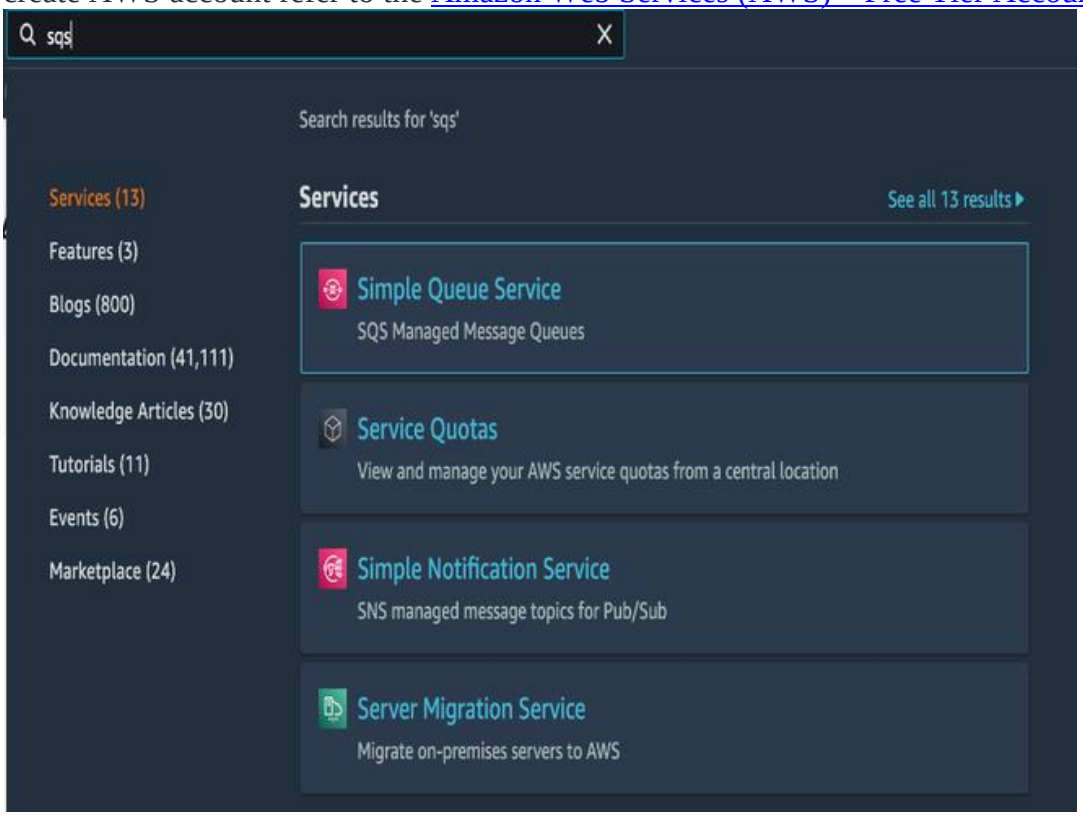




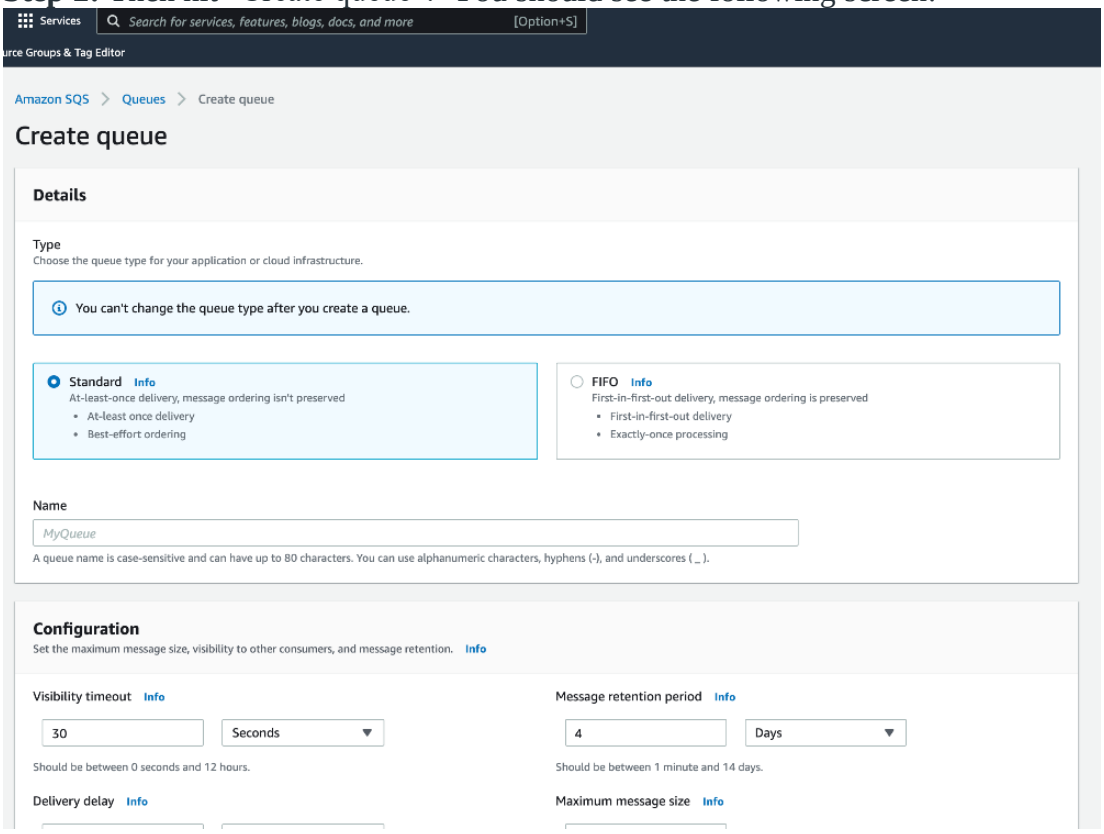
Life Cycle of AWS SQS

Step-By-Step Explanation To SQS Queues (Amazon Simple Queue Service)
Follow the below steps to create a SQS using the AWS console:

Step 1: Open AWS console and type “SQS” in the search bar. And select it.To know how to create AWS account refer to the [Amazon Web Services (AWS) – Free Tier Account Set up.]

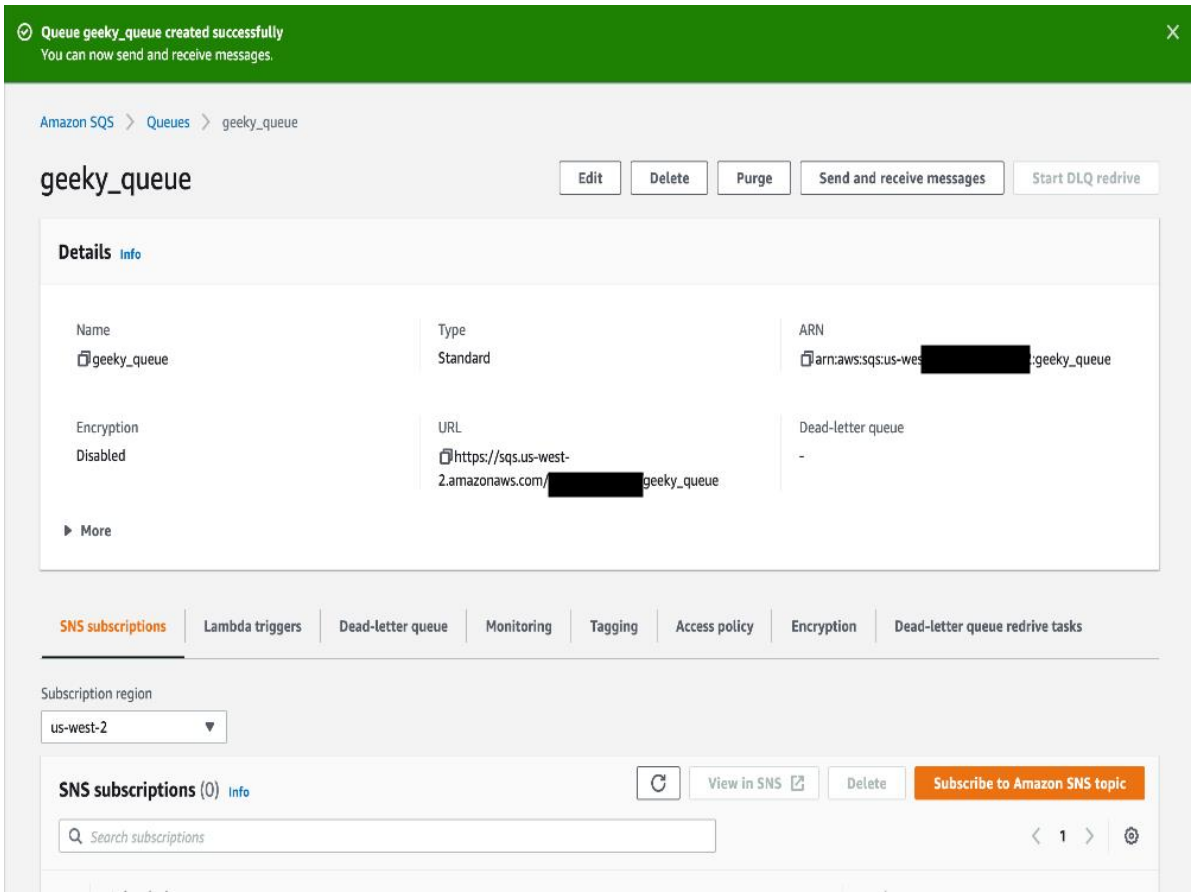


Step 2: Then hit “Create queue”. You should see the following screen:



Every field has an **Info** clause. You can click on it and read about it if curious.

Step 3: Name the queue ‘geeky_queue’ name and hit “Create Queue”. For other fields defaults are fine.



Deploying a simple NodeJS application with EC2

Amazon EC2 is Amazon Elastic Cloud Computing. This is a service where Amazon provides us a server where we can deploy our application for absolute minimum cost and it takes a very little time to do so.



By providing the service of AWS EC2 amazon is giving us a easy way to launch our application without worrying about the bare metal hardware stuff and the networking part.

Linux AMI

An Amazon Machine Image (AMI) is a pre-configured virtual machine image, available in the Amazon Web Services (AWS) ecosystem, that is used to launch an instance



A Linux AMI comes pre-installed with a Linux operating system. Some of the most popular Linux distributions available AMIs on AWS are Ubuntu, Amazon Linux.
So the steps that we need to perform here are as follows:

1. Create a AWS EC2 instance
2. SSH into the instance
3. Install NodeJS and NPM
4. Install Git
5. Clone the repository from GitHub
6. Install all the required dependencies
7. Run the application
8. Access the application in browser

Now we will start performing all these steps one by one :

1. Create a AWS EC2 instance

As a very first step here in the process we need to configure a AWS EC2 instance and SSH into the same.
To know the process of configuring a EC2 instance please check my previous article on the same.
Here is the Link : "Launching your first EC2 Instance"
Now we will start the procedure of deploying the Node.js application on the newly created EC2 machine.

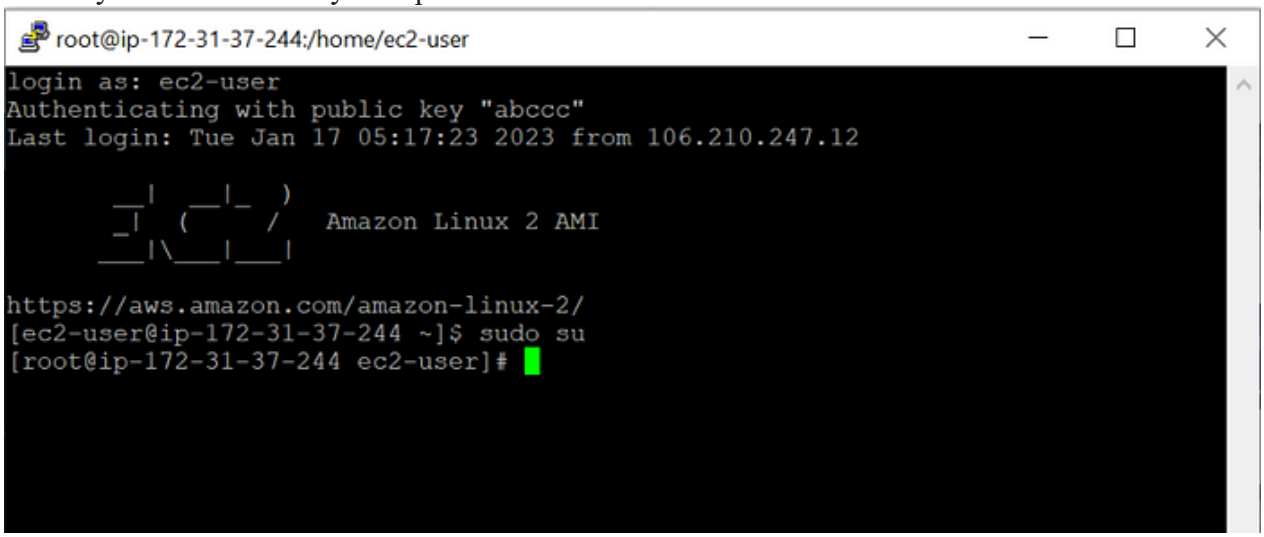
2. SSH into the instance

The steps to SSH into the EC2 machine has been explained in the previous article. Please check the link provided in 1st point.

After doing SSH into the EC2 instance you need to change the user to the super user by using the command :

```
#sudo su
```

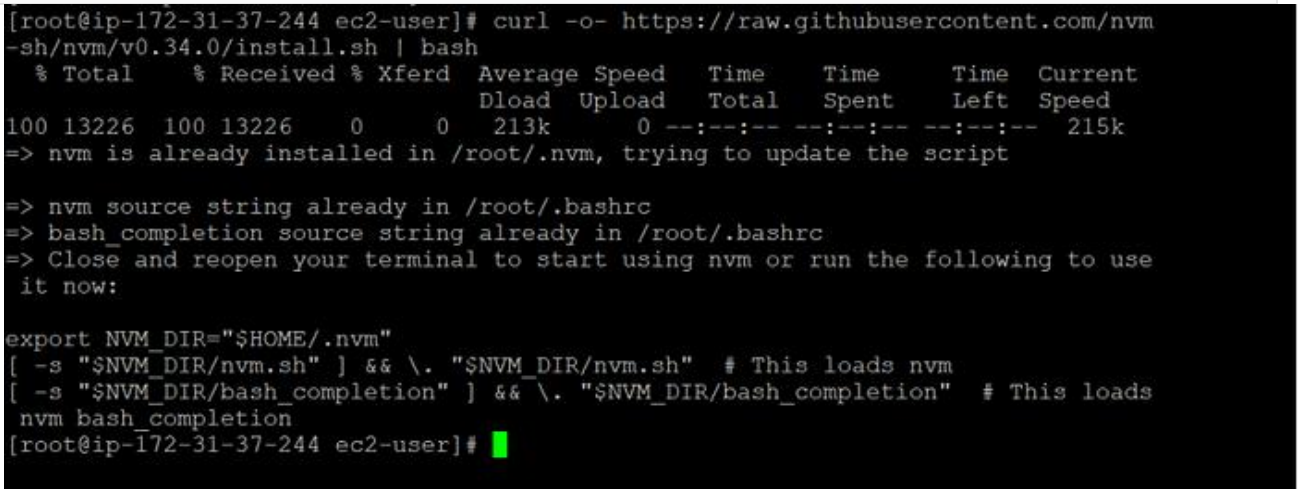
so that you won't have any user permission issues.



3. Install NodeJS and NPM using nvm

Now we need to install nvm (node version manager)by typing the following at the command line.

```
#curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
```



Now you need to activate nvm by using the command :

```
#. ~/.nvm/nvm.sh
```

By using nvm you have to install Node.js. Put the below command in the command line :

#nvm install 16

```
[root@ip-172-31-37-244 ec2-user]# . ~/.nvm/nvm.sh
[root@ip-172-31-37-244 ec2-user]# nvm install node
Downloading and installing node v19.4.0...
Downloading https://nodejs.org/dist/v19.4.0/node-v19.4.0-linux-x64.tar.xz...
################################################################################ 100.0%
Computing checksum with sha256sum
Checksums matched!
node: /lib64/libm.so.6: version `GLIBC_2.27' not found (required by node)
node: /lib64/libc.so.6: version `GLIBC_2.28' not found (required by node)
nvm is not compatible with the npm config "prefix" option: currently set to ""
Run `nvm use --delete-prefix v19.4.0` to unset it.
[root@ip-172-31-37-244 ec2-user]#
```

Check the nvm and Node.js version by using the commands

#nvm --version

```
[root@ip-172-31-37-244 ec2-user]# nvm --version
0.34.0
[root@ip-172-31-37-244 ec2-user]#
```

```
root@ip-172-31-37-244:/usr/local/bin
[root@ip-172-31-37-244 bin]# nvm install node
v19.4.0 is already installed.
```

4. Install Git

Now we need to take the source code of the Node.js application from the GitHub so we have to install Git first so that we can clone the particular repository of the Node.js application.
The #yum update -y command is used to update the existing software packages on the machine.
Use the below command to install Git on your machine :

#yum update -y
#hj

```
root@ip-172-31-37-244:/usr/local/bin
[root@ip-172-31-37-244 bin]# yum install update -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core                                               | 3.7 kB     00:00
amzn2extra-docker                                        | 3.0 kB     00:00
amzn2extra-java-openjdk11                                | 3.0 kB     00:00
amzn2extra-kernel-5.10                                   | 3.0 kB     00:00
jenkins                                                  | 2.9 kB     00:00
No package update available.
Error: Nothing to do
[root@ip-172-31-37-244 bin]#
```

```
[root@ip-172-31-37-244 bin]# yum install git -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
Resolving Dependencies
--> Running transaction check
---> Package git.x86_64 0:2.38.1-1.amzn2.0.1 will be installed
--> Processing Dependency: perl-Git = 2.38.1-1.amzn2.0.1 for package: git-2.38.1
-1.amzn2.0.1.x86_64
--> Processing Dependency: git-core-doc = 2.38.1-1.amzn2.0.1 for package: git-2.
38.1-1.amzn2.0.1.x86_64
--> Processing Dependency: git-core = 2.38.1-1.amzn2.0.1 for package: git-2.38.1
-1.amzn2.0.1.x86_64
--> Processing Dependency: perl(Term::ReadKey) for package: git-2.38.1-1.amzn2.0
.1.x86_64
--> Processing Dependency: perl(Git::I18N) for package: git-2.38.1-1.amzn2.0.1.x
86_64
--> Processing Dependency: perl(Git) for package: git-2.38.1-1.amzn2.0.1.x86_64
--> Running transaction check
  Verifying  : git-core-2.38.1-1.amzn2.0.1.x86_64
  Verifying  : 1:perl-Error-0.17020-2.amzn2.noarch
  Verifying  : perl-Git-2.38.1-1.amzn2.0.1.noarch

Installed:
  git.x86_64 0:2.38.1-1.amzn2.0.1

Dependency Installed:
  git-core.x86_64 0:2.38.1-1.amzn2.0.1              git-core-doc.noarch 0:2.38.1-1.amzn2.0.1
  perl-Git.noarch 0:2.38.1-1.amzn2.0.1              perl-TermReadKey.x86_64 0:2.30-20.amzn2.

Complete!
[root@ip-172-31-37-244 bin]#
```

You can check the version of the Git by putting the command

```
#git --version
```

```
[root@ip-172-31-37-244 bin]# git --version
git version 2.38.1
```

5. Clone the repository from GitHub

After installing Git, now we will clone the repository of the Node.js application.

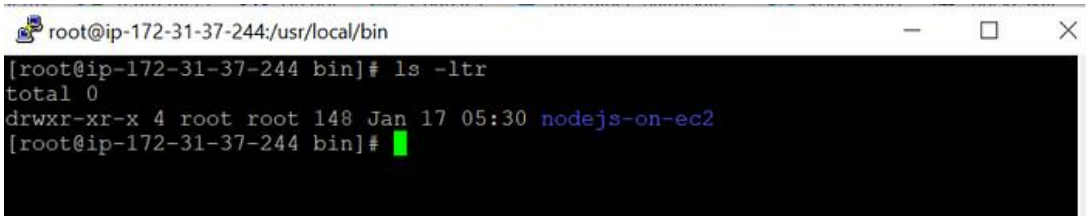
REPO-LINK :

https://github.com/rajani103/nodejs-on-ec2

Use the below command to do so :

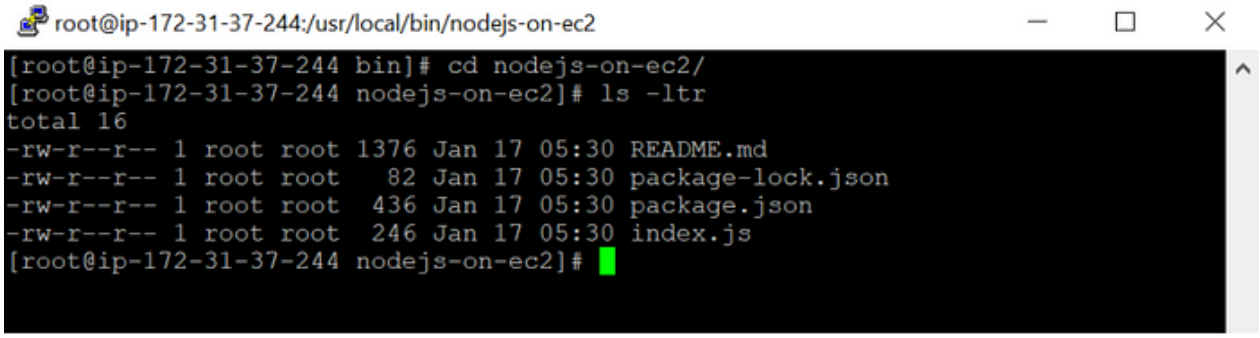
```
[root@ip-172-31-37-244 bin]# git clone https://github.com/rajani103/nodejs-on-ec
2.git
Cloning into 'nodejs-on-ec2'...
remote: Enumerating objects: 68, done.
remote: Total 68 (delta 0), reused 0 (delta 0), pack-reused 68
Receiving objects: 100% (68/68), 11.15 KiB | 2.79 MiB/s, done.
Resolving deltas: 100% (27/27), done.
[root@ip-172-31-37-244 bin]#
```

This command will clone(creating a copy) the repository on the machine by creating a folder by the name “nodejs-on-ec2”.



Now get into that folder and check the sub folders and files of the Node.js application. Use the command to check the files:

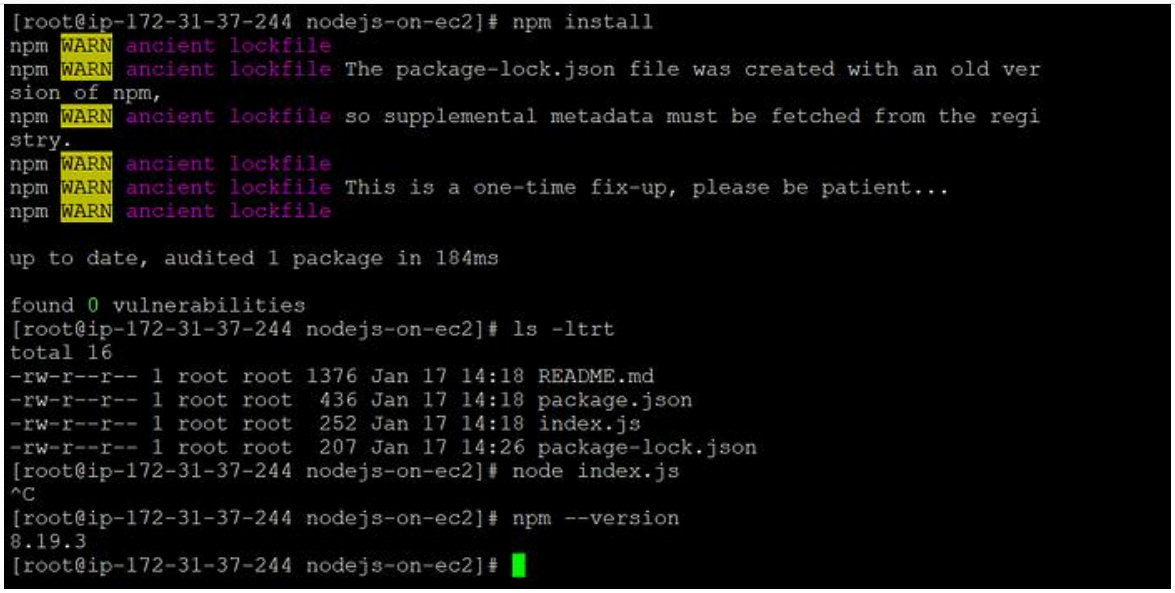
```
#cd nodejs-on-ec2
#ls -ltr
```



6. Install all the required dependencies

To install the dependencies use the below command :

```
#npm install
```



7. Run the application

To run the application put the below command in the command line :

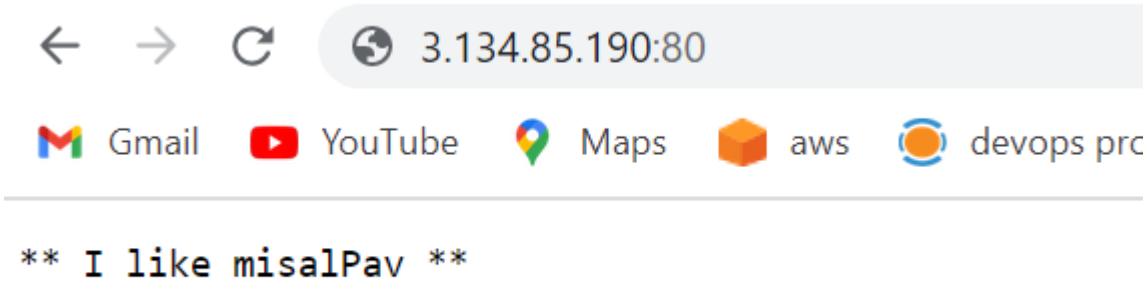
```
#node index.js
```

8. Access the application in browser

As you can see in the repository on the GitHub in the index.js file we have already mentioned to create a http server which is listening on port 80.
So there is no need to doing the port mapping again, as we can easily access the application by using the public IP of the machine and port number 80.
If everything till now is working fine then we will be able to see the application running on the port number 80.

```
3.134.85.190:80
(public IP of the machine : port 80)
```

You can see the browser output below :



You can also use the public DNS of the machine to get the access .

```
http://ec2-3-134-85-190.us-east-2.compute.amazonaws.com/:80
```

You will get the similar output.